

Contents

Módulo 4 - Práctica: persistencia de datos con JPA / Hibernate	3
Recorridos de entorno	3
Consola M4	3
IntelliJ IDEA M4	3
Eclipse M4	3
VS Code M4	3
Práctica 4.1 - Introducción a JPA e Hibernate	4
Paso 1 - Añadir las dependencias al pom.xml	4
Paso 2 - Configurar application.properties	4
Paso 3 - Arrancar y verificar la conexión	5
Paso 4 - Crear la entidad Alumno	5
Paso 5 - Arrancar y ver la tabla creada	6
Paso 6 - Crear el repositorio Spring Data JPA	7
Paso 7 - Modificar el servicio para usar la entidad	7
Paso 8 - Modificar el controlador	9
Paso 9 - Crear un CommandLineRunner para datos iniciales	10
Paso 10 - Arrancar y probar	11
Paso 11 - Verificar la persistencia	12
Paso 12 - Errores comunes del ejercicio	12
Paso 13 - Reto resuelto — Consulta personalizada con @Query	13
Resultado esperado global	13
Resumen técnico	14
Conclusión	14
Práctica 4.2 - Definición de entidades	14
Paso 1 - Repasar la entidad actual	14
Paso 2 - Añadir un enum de estado	15
Paso 3 - Añadir un campo calculado con @Transient	15
Paso 4 - Crear una clase embebida Direccion	16
Paso 5 - Añadir un campo de versión con @Version	17
Paso 6 - Añadir un campo @Lob para observaciones	17
Paso 7 - Arrancar y ver el SQL generado	17
Paso 8 - Actualizar el DTO y el servicio	17
Paso 9 - Arrancar y probar	18
Paso 10 - Ver el SQL de inserción	19
Paso 11 - Añadir equals y hashCode	19
Paso 12 - Errores comunes del ejercicio	20
Paso 13 - Reto resuelto — Añadir un campo @Lob binario	20
Resultado esperado global	21
Resumen técnico	21
Conclusión	21
Práctica 4.3 - Repositorios JPA	22
Paso 1 - Repasar el repositorio actual	22
Paso 2 - Añadir métodos derivados	22
Paso 3 - Arrancar y verificar	23
Paso 4 - Añadir un método con @Query	23
Paso 5 - Añadir un método con @Modifying	24
Paso 6 - Usar el método @Modifying desde el servicio	24
Paso 7 - Añadir paginación	24
Paso 8 - Probar la paginación	25
Paso 9 - Añadir Specifications	25
Paso 10 - Crear una clase de Specifications	26
Paso 11 - Usar Specifications en el servicio	26

Paso 12 - Probar Specifications	27
Paso 13 - Errores comunes del ejercicio	28
Paso 14 - Reto resuelto — Consulta con agregación	28
Resultado esperado global	28
Resumen técnico	29
Conclusión	29
Práctica 4.4 - Relaciones entre entidades: OneToMany, ManyToOne	29
Paso 1 - Crear la entidad Curso	30
Paso 2 - Modificar la entidad Alumno para añadir la relación	31
Paso 3 - Arrancar y ver el SQL generado	31
Paso 4 - Crear el repositorio de Curso	32
Paso 5 - Crear el DTO de Curso	32
Paso 6 - Crear el servicio de Curso	33
Paso 7 - Modificar el AlumnoService para asignar el curso	34
Paso 8 - Crear el controlador de Curso	34
Paso 9 - Añadir el método listarPorCurso al AlumnoService	35
Paso 10 - Arrancar y probar	36
Paso 11 - Detectar y resolver el problema N+1	36
Paso 12 - Errores comunes del ejercicio	37
Paso 13 - Reto resuelto — Eliminar un curso con cascade	37
Resultado esperado global	38
Resumen técnico	38
Conclusión	38
Práctica 4.5 - Consultas básicas	39
Paso 1 - Repasar el repositorio actual	39
Paso 2 - Consulta con JOIN simple	39
Paso 3 - Consulta con JOIN FETCH	39
Paso 4 - Consulta con función de string	39
Paso 5 - Consulta con función de fecha	40
Paso 6 - Consulta con agregación	40
Paso 7 - Consulta con HAVING	40
Paso 8 - Consulta con proyección a DTO	40
Paso 9 - Añadir los métodos al servicio	41
Paso 10 - Añadir los endpoints	42
Paso 11 - Probar las consultas	42
Paso 12 - Errores comunes del ejercicio	42
Paso 13 - Reto resuelto — Consulta con subconsulta	43
Resultado esperado global	43
Resumen técnico	44
Conclusión	44
Práctica 4.6 - Gestión de transacciones	44
Paso 1 - Repasar las anotaciones actuales	44
Paso 2 - Provocar un rollback	45
Paso 3 - Probar el rollback	45
Paso 4 - Quitar el rollback temporal	46
Paso 5 - Añadir @Transactional a un nuevo método con REQUIRES_NEW	46
Paso 6 - Usar el servicio de auditoría desde el servicio de alumnos	47
Paso 7 - Provocar un rollback y verificar que la auditoría persiste	47
Paso 8 - Probar el rollback con auditoría	48
Paso 9 - Quitar el rollback temporal	48
Paso 10 - Añadir bloqueo optimista	48
Paso 11 - Probar el bloqueo optimista	48
Paso 12 - Errores comunes del ejercicio	49
Paso 13 - Reto resuelto — Transacción con timeout	49

Resultado esperado global	50
Resumen técnico	50
Conclusión	50
Práctica 4.7 - Testing de JPA con @DataJpaTest	51
Paso 1 - Verificar la dependencia de test	51
Paso 2 - Crear la estructura de paquetes de test	51
Paso 3 - Crear el test básico de AlumnoRepository	51
Paso 4 - Probar el método findByCurso	52
Paso 5 - Probar el caso vacío	53
Paso 6 - Probar el método existsByDni	53
Paso 7 - Probar el constraint unique del DNI	54
Paso 8 - Probar una consulta con @Query	54
Paso 9 - Probar la relación entre Alumno y Curso	55
Paso 10 - Crear el test de CursoRepository	55
Paso 11 - Ejecutar los tests	56
Paso 12 - Errores comunes del ejercicio	57
Paso 13 - Reto resuelto — Test para Specification	57
Resultado esperado global	58
Resumen técnico	58
Conclusión	58

Módulo 4 - Práctica: persistencia de datos con JPA / Hibernate

Regla de ejecución: cada paso debe dejar un observable verificable. Los cambios temporales para provocar errores se revierten expresamente antes de avanzar al siguiente ejercicio.

Continuidad con M3: el proyecto acumulativo mantiene AlumnoRequestDTO para entrada y AlumnoResponseDTO para salida. Cuando la fuente original hablaba de un único AlumnoDTO, los fragmentos de M4 se adaptan a esa separación sin perder el concepto JPA que se está practicando.

Recorridos de entorno

Consola M4

Ejecuta `./mvnw` en Linux/macOS o `mvnw.cmd` en Windows desde `M4/proyecto`. Usa `curl -i`, los logs SQL y la consola H2 sólo cuando el paso lo indique.

IntelliJ IDEA M4

Importa `M4/proyecto/pom.xml` como proyecto Maven, selecciona Java 17 y ejecuta `MiProyectoApplication` o los tests sin sustituir el Maven Wrapper del proyecto.

Eclipse M4

Importa `M4/proyecto` como Existing Maven Project, selecciona Java 17 y conserva los perfiles y propiedades definidos en el proyecto.

VS Code M4

Abre `M4/proyecto`, usa las extensiones Java/Spring y ejecuta siempre el Maven Wrapper incluido para reproducir los mismos comandos que en consola y CI.

Práctica 4.1 - Introducción a JPA e Hibernate

Contexto del ejercicio: Vamos a convertir el repositorio en memoria de alumnos en un repositorio real con JPA y H2. Veremos cómo Spring Boot configura la base de datos automáticamente, cómo definir una entidad, cómo crear un repositorio de Spring Data JPA, y cómo verificar que los datos persisten entre reinicios. Requisitos previos: Tener el proyecto mi-proyecto con el AlumnoController, AlumnoService y AlumnoRepository del Módulo 2.

Paso 1 - Añadir las dependencias al pom.xml

Abre el pom.xml y añade las dependencias de JPA y H2:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

La primera trae JPA, Hibernate y Spring Data JPA. La segunda trae el driver de H2.

Guarda el pom.xml y recarga Maven:

- IntelliJ: clic derecho en el pom.xml > Maven > Reload Project.
- Eclipse: clic derecho en el proyecto > Maven > Update Project.
- VS Code: Ctrl+Shift+P > Java: Reload Projects.

Pregunta de reflexión: ¿Por qué H2 tiene <scope>runtime</scope>? ¿Qué significa eso?

Paso 2 - Configurar application.properties

Abre src/main/resources/application.properties y añade la configuración de H2:

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.h2.console.enabled=true
```

Cada línea configura un aspecto de la conexión:

- spring.datasource.url: la URL de conexión. jdbc:h2:mem:testdb es una base de datos H2 en memoria llamada testdb.
- spring.datasource.driver-class-name: el driver JDBC de H2.
- spring.datasource.username y password: credenciales. H2 usa sa sin contraseña por defecto.
- spring.jpa.database-platform: el dialecto de Hibernate para H2.
- spring.jpa.hibernate.ddl-auto=create-drop: crea las tablas al arrancar y las borra al parar. Perfecto para desarrollo.
- spring.jpa.show-sql=true: muestra el SQL en los logs. Útil para ver qué está haciendo Hibernate.
- spring.h2.console.enabled=true: habilita la consola web de H2 en /h2-console.

Pregunta de reflexión: ¿Qué diferencia hay entre create-drop y update? ¿Cuándo usarías cada uno?

Paso 3 - Arrancar y verificar la conexión

Arranca la aplicación. Si todo va bien, verás en los logs algo como:

```
HikariPool-1 - Starting...
HikariPool-1 - Start completed.
HHH000412: Hibernate ORM core version 6.4.0.Final
...
Tomcat started on port(s): 8080 (http)
Started MiProyectoApplication in X.XXX seconds
```

Las líneas de HikariCP confirman que el pool de conexiones se ha creado. Hibernate se ha inicializado. La aplicación está lista.

Ahora abre la consola de H2 en el navegador: <http://localhost:8080/h2-console>.

- JDBC URL: jdbc:h2:mem:testdb
- User Name: sa
- Password: (vacío)

Haz clic en Connect. Verás una interfaz web con un explorador de la base de datos. A la izquierda aparecen las tablas. De momento no hay ninguna, porque no hemos definido entidades.

Pregunta de reflexión: ¿Qué ventaja tiene la consola de H2 para el desarrollo?

Paso 4 - Crear la entidad Alumno

Vamos a crear la entidad Alumno en el paquete alumno. Si has reorganizado el proyecto por funcionalidad, el paquete ya existe. Si no, créalo.

```
package es.mecd.demo.miproyecto.alumno;
import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
@Table(name = "alumnos")
public class Alumno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String nombre;

    @Column(nullable = false, length = 150)
    private String apellidos;

    @Column(nullable = false, unique = true, length = 9)
    private String dni;

    @Column(name = "fecha_nacimiento")
    private LocalDate fechaNacimiento;

    @Column(nullable = false, length = 50)
    private String curso;
    public Alumno() {
    }

    public Alumno(
```

```

        String nombre,
        String apellidos,
        String dni,
        LocalDate fechaNacimiento,
        String curso) {
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.dni = dni;
        this.fechaNacimiento = fechaNacimiento;
        this.curso = curso;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public String getApellidos() { return apellidos; }
    public void setApellidos(String apellidos) { this.apellidos = apellidos; }

    public String getDni() { return dni; }
    public void setDni(String dni) { this.dni = dni; }

    public LocalDate getFechaNacimiento() { return fechaNacimiento; }
    public void setFechaNacimiento(LocalDate fechaNacimiento) {
        this.fechaNacimiento = fechaNacimiento;
    }

    public String getCurso() { return curso; }
    public void setCurso(String curso) { this.curso = curso; }
}

```

Los detalles clave:

- @Entity marca la clase como entidad. Hibernate la mapeará a una tabla.
- @Table(name = "alumnos") indica el nombre de la tabla. Si no se pone, se usaría alumno (el nombre de la clase en minúsculas).
- @Id marca el campo id como clave primaria.
- @GeneratedValue(strategy = GenerationType.IDENTITY) indica que el ID se genera automáticamente por la base de datos (autoincrement).
- @Column personaliza cada columna: nombre, longitud, si acepta nulos, si es único.
- El constructor sin argumentos es obligatorio para JPA.
- El constructor con argumentos es una comodidad para crear instancias.

Pregunta de reflexión: ¿Por qué el campo dni tiene unique = true? ¿Qué garantiza eso?

Paso 5 - Arrancar y ver la tabla creada

Reinicia la aplicación. Ahora, en los logs, verás que Hibernate ha creado la tabla:

```

Hibernate: create table alumnos (
  id bigint generated by default as identity,
  apellidos varchar(150) not null,
  curso varchar(50) not null,
  dni varchar(9) not null,
  fecha_nacimiento date,
  nombre varchar(100) not null,

```

```

    primary key (id)
)
Hibernate: alter table if exists alumnos
    add constraint UK_... unique (dni)

```

Hibernate ha generado el SQL de creación de la tabla a partir de la entidad. La columna id es bigint con autoincrement. Las columnas nombre, apellidos, curso y dni son not null. La columna dni tiene una restricción unique.

Abre la consola de H2 (<http://localhost:8080/h2-console>) y verás la tabla ALUMNOS en la lista. Puedes hacer clic en ella para ver su estructura y ejecutar consultas SQL.

Pregunta de reflexión: ¿Qué ha pasado con los datos que teníamos en el repositorio en memoria? ¿Siguen ahí?

Paso 6 - Crear el repositorio Spring Data JPA

Vamos a crear el repositorio. Elimina la clase AlumnoRepository en memoria y crea una interfaz:

```

package es.mecd.demo.miproyecto.alumno;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface AlumnoRepository extends JpaRepository<Alumno, Long> {

    Optional<Alumno> findByDni(String dni);

    boolean existsByDni(String dni);
}

```

JpaRepository<Alumno, Long> indica que el repositorio gestiona entidades Alumno con ID de tipo Long. Spring genera la implementación automáticamente.

findByDni y existsByDni son métodos derivados. Spring Data JPA genera la consulta a partir del nombre. Elimina la clase AlumnoRepository en memoria (la que tenía el Map). Si la dejas, habrá dos beans del mismo tipo y Spring fallará.

Pregunta de reflexión: ¿Qué diferencia hay entre el repositorio en memoria y el repositorio Spring Data JPA?

Paso 7 - Modificar el servicio para usar la entidad

El servicio ya no trabaja con AlumnoResponseDTO en el repositorio, sino con Alumno (la entidad). Hay que modificar el AlumnoService para transformar entre DTO y entidad.

```

package es.mecd.demo.miproyecto.alumno;

import es.mecd.demo.miproyecto.common.exception.NegocioException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.Optional;

@Service
public class AlumnoService {
    private final AlumnoRepository repositorio;

    public AlumnoService(AlumnoRepository repositorio) {

```

```
        this.repositorio = repositorio;
    }

    @Transactional(readOnly = true)
    public List<AlumnoResponseDTO> listar() {
        return repositorio.findAll().stream()
            .map(this::toDTO)
            .toList();
    }

    @Transactional(readOnly = true)
    public Optional<AlumnoResponseDTO> consultar(Long id) {
        return repositorio.findById(id).map(this::toDTO);
    }

    @Transactional
    public AlumnoResponseDTO crear(AlumnoRequestDTO request) {
        if (repositorio.existsByDni(request.getDni())) {
            throw new NegocioException(
                "Ya existe un alumno con el DNI " + request.getDni());
        }
        Alumno entidad = toEntity(request);
        Alumno guardado = repositorio.save(entidad);
        return toDTO(guardado);
    }

    @Transactional
    public Optional<AlumnoResponseDTO> actualizar(
        Long id,
        AlumnoRequestDTO request) {
        return repositorio.findById(id).map(entidad -> {
            entidad.setNombre(request.getNombre());
            entidad.setApellidos(request.getApellidos());
            entidad.setDni(request.getDni());
            entidad.setFechaNacimiento(request.getFechaNacimiento());
            entidad.setCurso(request.getCurso());
            return toDTO(repositorio.save(entidad));
        });
    }

    @Transactional
    public boolean eliminar(Long id) {
        if (!repositorio.existsById(id)) {
            return false;
        }
        repositorio.deleteById(id);
        return true;
    }

    private AlumnoResponseDTO toDTO(Alumno entidad) {
        AlumnoResponseDTO dto = new AlumnoResponseDTO();
        dto.setIdentificador(
            entidad.getId() != null
                ? entidad.getId().toString()
                : null);
        dto.setNombre(entidad.getNombre());
    }
}
```

```

        dto.setApellidos(entidad.getApellidos());
        dto.setDni(entidad.getDni());
        dto.setFechaNacimiento(entidad.getFechaNacimiento());
        dto.setCurso(entidad.getCurso());
        return dto;
    }

    private Alumno toEntity(AlumnoRequestDTO request) {
        return new Alumno(
            request.getNombre(),
            request.getApellidos(),
            request.getDni(),
            request.getFechaNacimiento(),
            request.getCurso()
        );
    }
}

```

Fíjate en los cambios:

- El ID ahora es Long, no String. La base de datos genera IDs numéricos.
- @Transactional en los métodos que modifican datos. @Transactional(readonly = true) en las consultas.
- toDTO y toEntity transforman entre la entidad y los DTOs.
- repositorio.save() inserta o actualiza. Si la entidad tiene ID, actualiza; si no, inserta.
- repositorio.deleteById() elimina por ID.

Pregunta de reflexión: ¿Por qué el servicio ahora tiene @Transactional? ¿Qué pasaría si no lo tuviera?

Paso 8 - Modificar el controlador

El controlador tiene que usar el nuevo tipo de ID (Long en lugar de String):

```

@RestController
@RequestMapping("/api/v1/alumnos")
public class AlumnoController {

    private final AlumnoService service;

    public AlumnoController(AlumnoService service) {
        this.service = service;
    }

    @GetMapping
    public List<AlumnoResponseDTO> listar() {
        return service.listar();
    }

    @GetMapping("/{id}")
    public ResponseEntity<AlumnoResponseDTO> consultar(@PathVariable Long id) {
        return service.consultar(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<AlumnoResponseDTO> crear(
        @Valid @RequestBody AlumnoRequestDTO request) {

```

```

        AlumnoResponseDTO creado = service.crear(request);
        URI location = ServletUriComponentsBuilder
            .fromCurrentRequest()
            .path("/{id}")
            .buildAndExpand(creado.getIdentificador())
            .toUri();
        return ResponseEntity.created(location).body(creado);
    }

    @PutMapping("/{id}")
    public ResponseEntity<AlumnoResponseDTO> actualizar(
        @PathVariable Long id,
        @Valid @RequestBody AlumnoRequestDTO request) {
        return service.actualizar(id, request)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminar(@PathVariable Long id) {
        return service.eliminar(id)
            ? ResponseEntity.noContent().build()
            : ResponseEntity.notFound().build();
    }
}

```

El cambio principal es que @PathVariable ahora es Long en lugar de String. El resto es igual.

Pregunta de reflexión: ¿Qué pasa si el cliente envía un ID no numérico? ¿Qué código de estado devuelve Spring?

Paso 9 - Crear un CommandLineRunner para datos iniciales

Para tener datos con los que probar, vamos a crear un CommandLineRunner que inserte algunos alumnos al arrancar:

```

package es.mecd.demo.miproyecto.config;

import es.mecd.demo.miproyecto.alumno.Alumno;
import es.mecd.demo.miproyecto.alumno.AlumnoRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.time.LocalDate;

@Configuration
public class DatosInicialesConfig {

    @Bean
    public CommandLineRunner cargarAlumnos(AlumnoRepository repositorio) {
        return args -> {
            if (repositorio.count() == 0) {
                repositorio.save(new Alumno("Ana", "García López", "12345678A",
                    LocalDate.of(2010, 5, 12), "5º Primaria"));
                repositorio.save(new Alumno("Luis", "Pérez Ruiz", "87654321B",
                    LocalDate.of(2009, 9, 3), "6º Primaria"));
            }
        }
    }
}

```

```

    };
}
}

```

`repositorio.count() == 0` comprueba si la tabla está vacía antes de insertar. Así, si la persistencia se conserva entre reinicios, no se duplican los datos iniciales.

Pregunta de reflexión: ¿Por qué se comprueba `count() == 0` antes de insertar? ¿Qué pasaría si no se comprobara?

Paso 10 - Arrancar y probar

Reinicia la aplicación. En los logs verás el SQL de inserción:

```

Hibernate: insert into alumnos
  (apellidos, curso, dni, fecha_nacimiento, nombre, id)
  values (?, ?, ?, ?, ?, default)
Hibernate: insert into alumnos
  (apellidos, curso, dni, fecha_nacimiento, nombre, id)
  values (?, ?, ?, ?, ?, default)

```

Ahora prueba los endpoints con curl:

Listar

```
curl http://localhost:8080/api/v1/alumnos
```

Consultar

```
curl http://localhost:8080/api/v1/alumnos/1
```

Crear

```

curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{
    "nombre": "María",
    "apellidos": "López",
    "dni": "11111111C",
    "fechaNacimiento": "2011-03-20",
    "curso": "4º Primaria"
  }'

```

Actualizar

```

curl -i -X PUT http://localhost:8080/api/v1/alumnos/1 \
  -H "Content-Type: application/json" \
  -d '{
    "nombre": "Ana",
    "apellidos": "García López",
    "dni": "12345678A",
    "fechaNacimiento": "2010-05-12",
    "curso": "6º Primaria"
  }'

```

Eliminar

```
curl -i -X DELETE http://localhost:8080/api/v1/alumnos/3
```

Los endpoints funcionan igual que antes, pero ahora los datos van a la base de datos.

Pregunta de reflexión: ¿Qué diferencia hay entre los logs de antes (con el repositorio en memoria) y los de ahora (con JPA)?

Paso 11 - Verificar la persistencia

Con la configuración actual (create-drop), los datos se borran al parar la aplicación. Vamos a cambiar a update para que persistan:

```
spring.jpa.hibernate.ddl-auto=update
```

Reinicia la aplicación. Ahora los datos no se borran al parar. Crea un alumno, para la aplicación, vuelve a arrancarla y consulta el alumno. Sigue ahí.

Nota: H2 en memoria (jdbc:h2:mem:testdb) pierde los datos al parar la JVM. Para que persistan entre reinicios, hay que usar H2 en fichero (jdbc:h2:file:./data/testdb) o PostgreSQL.

Cambia la URL a fichero:

```
spring.datasource.url=jdbc:h2:file:./data/testdb
```

Reinicia. Ahora los datos se guardan en un fichero data/testdb.mv.db. Para la aplicación, vuelve a arrancarla y verifica que los datos siguen ahí.

Cierre obligatorio del cambio temporal. Esta configuración se usa sólo para observar persistencia entre reinicios. Antes de continuar con el paso 12, restaura el perfil de desarrollo al estado canónico del módulo:

```
spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE
spring.jpa.hibernate.ddl-auto=create-drop
```

Detén la aplicación, elimina data/testdb.mv.db si se creó durante la prueba y vuelve a ejecutar ./mvnw test. El observable de cierre es que la suite queda verde con la configuración canónica restaurada. Esta transición MODIFY -> VERIFY -> RESTORE también queda registrada en la trazabilidad de M4.

Pregunta de reflexión: ¿Qué diferencia hay entre H2 en memoria y H2 en fichero? ¿Por qué conviene restaurar la configuración de laboratorio antes de seguir?

Paso 12 - Errores comunes del ejercicio

Error	Causa	Solución
Table "ALUMNOS" not found	ddl-auto=none o entidad mal configurada	Usar create-drop/update en desarrollo y verificar el escaneo de entidades.
Unique index or primary key violation	DNI duplicado	Validar antes de guardar y conservar UNIQUE.
DataIntegrityViolationException	Campo NOT NULL sin valor	Validar la entrada antes de persistir.
LazyInitializationException	Relación usada fuera de transacción	Transformar a DTO dentro del servicio.
No default constructor for entity	Falta constructor sin argumentos	Añadir constructor público/protegido sin argumentos.
Could not determine type for column	Tipo no soportado/mal mapeado	Revisar anotaciones JPA.
NoSuchBeanDefinitionException	Repositorio no detectado	Extender JpaRepository y ubicarlo bajo el paquete raíz.
Puerto 8080 ocupado	Otro proceso usa el puerto	Pararlo o cambiar el puerto.
Consola H2 inaccesible	No está habilitada	Activarla sólo en desarrollo.
Connection refused	URL JDBC incorrecta/BD no disponible	Verificar datasource y perfil.

Paso 13 - Reto resuelto — Consulta personalizada con @Query

Reto: Añadir un método al repositorio que devuelva los alumnos de un curso nacidos después de una fecha.

Solución paso a paso:

Subpaso 1. Añadir el método al repositorio:

```
@Query(
    "SELECT a FROM Alumno a "
    + "WHERE a.curso = :curso AND a.fechaNacimiento > :fecha"
)
List<Alumno> buscarPorCursoYNacidosDespues(
    @Param("curso") String curso,
    @Param("fecha") LocalDate fecha);
```

Subpaso 2. Añadir el método al servicio:

```
@Transactional(readOnly = true)
public List<AlumnoResponseDTO> buscarPorCursoYNacidosDespues(
    String curso,
    LocalDate fecha) {
    return repositorio.buscarPorCursoYNacidosDespues(curso, fecha).stream()
        .map(this::toDTO)
        .toList();
}
```

Subpaso 3. Añadir el endpoint al controlador:

```
@GetMapping("/buscar")
public List<AlumnoResponseDTO> buscar(
    @RequestParam String curso,
    @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate fecha) {
    return service.buscarPorCursoYNacidosDespues(curso, fecha);
}
```

Subpaso 4. Probar:

```
curl "http://localhost:8080/api/v1/alumnos/buscar" \
  --get \
  --data-urlencode "curso=5º Primaria" \
  --data-urlencode "fecha=2010-01-01"
```

Devuelve los alumnos de 5º Primaria nacidos después del 1 de enero de 2010. > **Pregunta de reflexión:** ¿Qué diferencia hay entre findByCurso (derivado) y @Query? ¿Cuándo usarías cada uno?

Resultado esperado global

Al final del ejercicio, deberías tener:

- Un AlumnoRepository que extiende JpaRepository.
- Una entidad Alumno con anotaciones JPA.
- Un AlumnoService con @Transactional y transformación DTO ↔ Entidad.
- Un AlumnoController que usa Long como tipo de ID.
- La aplicación configurada con H2 en memoria (o en fichero).
- Los endpoints funcionando y persistiendo datos.

Resumen técnico

El ejercicio ha demostrado: - JPA + Hibernate + Spring Data JPA: cómo se integran. - Dependencias: spring-boot-starter-data-jpa + driver. - Configuración: application.properties con H2. - Entidad: @Entity, @Id, @GeneratedValue, @Column. - Repositorio: interfaz que extiende JpaRepository. - Consultas derivadas: findByDni, existsByDni. - @Query: para consultas personalizadas. - Transacciones: @Transactional. - Transformación: DTO ↔ Entidad en el servicio. - Persistencia: H2 en memoria vs fichero.

Conclusión

En este punto 4.1 hemos introducido JPA, Hibernate y Spring Data JPA:

- Qué es JPA: especificación de Java para mapear objetos a tablas.
- Qué es Hibernate: implementación de referencia de JPA.
- Qué es Spring Data JPA: capa sobre JPA que simplifica el acceso a datos.
- Configuración: dependencias, application.properties, perfiles.
- Entidad: clase con @Entity, @Id, @Column.
- Repositorio: interfaz que extiende JpaRepository.
- Consultas derivadas y @Query.
- Transacciones: @Transactional.

En la práctica, hemos convertido el repositorio en memoria en un repositorio real con JPA y H2. Los datos ahora persisten en una base de datos, y los endpoints funcionan igual que antes pero con persistencia real.

La idea clave: JPA y Spring Data JPA eliminan la mayor parte del trabajo de acceso a datos. Tú defines la entidad y el repositorio; Spring y Hibernate hacen el resto.

En el siguiente punto, 4.2 – Definición de entidades, profundizaremos en cómo definir entidades JPA correctamente: tipos de datos, anotaciones avanzadas, generación de IDs, campos calculados, y buenas prácticas.

Fin del Punto 4.1.

Práctica 4.2 - Definición de entidades

Contexto del ejercicio: Vamos a ampliar la entidad Alumno con campos de distintos tipos, un enum, un campo calculado y una dirección embebida. Veremos cómo Hibernate genera el SQL y cómo se comporta la entidad en la base de datos.

Requisitos previos: Tener el proyecto mi-proyecto con JPA y H2 configurados (punto 4.1).

Paso 1 - Repasar la entidad actual

Abre la entidad Alumno y observa su contenido:

```
@Entity
@Table(name = "alumnos")
public class Alumno {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String nombre;

    @Column(nullable = false, length = 150)
    private String apellidos;
```

```

@Column(nullable = false, unique = true, length = 9)
private String dni;

@Column(name = "fecha_nacimiento")
private LocalDate fechaNacimiento;

@Column(nullable = false, length = 50)
private String curso;

// constructores, getters y setters
}

```

Esta entidad tiene los campos básicos. Vamos a ampliarla.

Pregunta de reflexión: ¿Qué campos crees que le faltan a un alumno en un sistema real?

Paso 2 - Añadir un enum de estado

Vamos a añadir un enum EstadoAlumno con los estados posibles:

```

package es.mecd.demo.miproyecto.alumno;

public enum EstadoAlumno {
    ACTIVO,
    INACTIVO,
    SUSPENDIDO,
    GRADUADO
}

```

Y añadimos el campo a la entidad:

```

@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 20)
private EstadoAlumno estado = EstadoAlumno.ACTIVO;

```

@Enumerated(EnumType.STRING) guarda el nombre del enum. @Column(nullable = false, length = 20) hace la columna obligatoria y acota su longitud. La inicialización con EstadoAlumno.ACTIVO proporciona el valor por defecto en nuevas instancias Java.

Pregunta de reflexión: ¿Qué pasaría si usáramos EnumType.ORDINAL y luego reordenáramos el enum?

Paso 3 - Añadir un campo calculado con @Transient

Vamos a añadir un campo edad que se calcula a partir de fechaNacimiento:

```

@Transient
private Integer edad;

public Integer getEdad() {
    if (fechaNacimiento == null) {
        return null;
    }
    return Period.between(fechaNacimiento, LocalDate.now()).getYears();
}

```

@Transient indica que el valor calculado no se persiste y, por tanto, no genera una columna.

Period.between(...) calcula el periodo entre dos fechas. getYears() devuelve los años.

Este campo se puede consultar desde el código, pero no se guarda en la base de datos. Cada vez que se llama a `getEdad()`, se calcula al vuelo.

Pregunta de reflexión: ¿Qué ventaja tiene calcular la edad al vuelo en lugar de guardarla en la base de datos?

Paso 4 - Crear una clase embebida Direccion

Vamos a crear una clase `Direccion` que se pueda embeber en la entidad:

```
package es.mecd.demo.miproyecto.alumno;

import jakarta.persistence.Column;
import jakarta.persistence.Embeddable;

@Embeddable
public class Direccion {

    @Column(name = "calle", length = 200)
    private String calle;
    @Column(name = "numero", length = 10)
    private String numero;

    @Column(name = "ciudad", length = 100)
    private String ciudad;

    @Column(name = "codigo_postal", length = 5)
    private String codigoPostal;

    public Direccion() {
    }

    public Direccion(
        String calle,
        String numero,
        String ciudad,
        String codigoPostal) {
        this.calle = calle;
        this.numero = numero;
        this.ciudad = ciudad;
        this.codigoPostal = codigoPostal;
    }

    // getters y setters
}
```

`@Embeddable` marca la clase como tipo de valor embebible. Sus campos se mapean en la tabla de la entidad propietaria; `@Column` personaliza cada columna.

Y añadimos el campo a la entidad `Alumno`:

```
@Embeddedd
private Direccion direccion;
```

`@Embeddedd` indica que `direccion` es un objeto embebido.

Pregunta de reflexión: ¿Qué diferencia hay entre `@Embeddedd` y una relación `@One-ToOne`?

Paso 5 - Añadir un campo de versión con @Version

Vamos a añadir un campo de versión para el bloqueo optimista:

```
@Version
private Integer version;
```

@Version implementa bloqueo optimista: Hibernate incrementa la versión y la incluye en el WHERE del UPDATE para detectar modificaciones concurrentes. > **Pregunta de reflexión:** ¿Qué pasa si dos transacciones intentan actualizar la misma entidad a la vez?

Paso 6 - Añadir un campo @Lob para observaciones

Vamos a añadir un campo para observaciones largas:

```
@Lob
@Column(name = "observaciones")
private String observaciones;
```

Con String, @Lob representa texto grande (habitualmente CLOB según dialecto).

Pregunta de reflexión: ¿Qué diferencia hay entre VARCHAR(255) y CLOB?

Paso 7 - Arrancar y ver el SQL generado

Reinicia la aplicación. En los logs verás el SQL de creación de la tabla:

```
Hibernate: create table alumnos (
  id bigint generated by default as identity,
  apellidos varchar(150) not null,
  calle varchar(200),
  ciudad varchar(100),
  codigo_postal varchar(5),
  curso varchar(50) not null,
  dni varchar(9) not null,
  estado varchar(20) not null,
  fecha_nacimiento date,
  nombre varchar(100) not null,
  numero varchar(10),
  observaciones clob,
  version integer,
  primary key (id)
)
```

Observa:

- estado varchar(20) not null: el enum se guarda como string.
- calle, ciudad, codigo_postal, numero: los campos de la dirección embebida se mapean a columnas de la tabla alumnos. No se crea una tabla direcciones.
- observaciones clob: el campo @Lob se mapea a CLOB.
- version integer: el campo de versión se mapea a INTEGER.
- No aparece edad: el campo @Transient no se mapea.

Pregunta de reflexión: ¿Qué ha pasado con el campo edad? ¿Por qué no aparece en el SQL?

Paso 8 - Actualizar el DTO y el servicio

Vamos a actualizar el DTO de salida para incluir los nuevos campos:

```

public class AlumnoResponseDTO {

    @JsonProperty("id")
    private String identificador;

    private String nombre;
    private String apellidos;
    private String dni;

    @JsonFormat(pattern = "yyyy-MM-dd")
    private LocalDate fechaNacimiento;
    private String curso;
    private String estado;
    private Integer edad;

    @JsonProperty("direccion")
    private DireccionDTO direccion;

    private String observaciones;

    // getters y setters
}

```

Y el servicio, para transformar entre entidad y DTO:

```

private AlumnoResponseDTO toDTO(Alumno entidad) {
    AlumnoResponseDTO dto = new AlumnoResponseDTO();
    dto.setIdentificador(
        entidad.getId() != null
            ? entidad.getId().toString()
            : null);
    dto.setNombre(entidad.getNombre());
    dto.setApellidos(entidad.getApellidos());
    dto.setDni(entidad.getDni());
    dto.setFechaNacimiento(entidad.getFechaNacimiento());
    dto.setCurso(entidad.getCurso());
    dto.setEstado(entidad.getEstado() != null ? entidad.getEstado().name() : null);
    dto.setEdad(entidad.getEdad());
    if (entidad.getDireccion() != null) {
        DireccionDTO direccionDTO = new DireccionDTO();
        direccionDTO.setCalle(entidad.getDireccion().getCalle());
        direccionDTO.setNumero(entidad.getDireccion().getNumero());
        direccionDTO.setCiudad(entidad.getDireccion().getCiudad());
        direccionDTO.setCodigoPostal(entidad.getDireccion().getCodigoPostal());
        dto.setDireccion(direccionDTO);
    }
    dto.setObservaciones(entidad.getObservaciones());
    return dto;
}

```

Pregunta de reflexión: ¿Por qué el campo estado se transforma con .name() en lugar de pasarlo directamente?

Paso 9 - Arrancar y probar

Reinicia la aplicación y prueba los endpoints:

```
# Listar
curl http://localhost:8080/api/v1/alumnos

# Crear con todos los campos
curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{
    "nombre": "María",
    "apellidos": "López Fernández",
    "dni": "11111111C",
    "fechaNacimiento": "2011-03-20",
    "curso": "4º Primaria",
    "estado": "ACTIVO",
    "direccion": {
      "calle": "Calle Mayor",
      "numero": "10",
      "ciudad": "Madrid",
      "codigoPostal": "28001"
    },
    "observaciones": "Alumno nuevo"
  }'
```

Verás que la respuesta incluye los nuevos campos, y el campo edad se calcula automáticamente.

Pregunta de reflexión: ¿Qué valor tiene el campo edad en la respuesta? ¿De dónde sale?

Paso 10 - Ver el SQL de inserción

En los logs de la aplicación verás el SQL de inserción:

```
Hibernate: insert into alumnos
  (apellidos, calle, ciudad, codigo_postal, curso, dni, estado,
   fecha_nacimiento, nombre, numero, observaciones, version, id)
values (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, default)
```

Observa que se incluyen todas las columnas, incluidas las de la dirección embebida. El campo edad no aparece porque es @Transient.

Pregunta de reflexión: ¿Qué columnas no aparecen en el INSERT? ¿Por qué?

Paso 11 - Añadir equals y hashCode

Vamos a añadir equals y hashCode a la entidad Alumno:

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Alumno)) return false;
    Alumno otro = (Alumno) o;
    return id != null && id.equals(otro.id);
}

@Override
public int hashCode() {
    return getClass().hashCode();
}
```

equals compara por ID. Si los IDs son null (entidades no persistidas), se consideran distintas.

hashCode devuelve el hash de la clase. Es estable a lo largo del ciclo de vida de la entidad, incluso cuando el ID se asigna.

Pregunta de reflexión: ¿Por qué el hashCode no usa el ID? ¿Qué problema evita?

Paso 12 - Errores comunes del ejercicio

Error	Causa	Solución
No default constructor for entity	Falta constructor sin argumentos	Añadirlo.
Could not determine type for column	Tipo no soportado	Revisar el mapeo.
Data too long for column	Longitud insuficiente	Ajustar @Column(length=...) o tipo SQL.
Columna obligatoria nula	nullable=false sin valor	Asignar valor antes de persistir.
LazyInitializationException	Relación LAZY fuera de transacción	Resolver dentro del servicio.
OptimisticLockException	Conflicto de versiones	Informar/reintentar de forma controlada.
Enum incompatible	Uso de ORDINAL	Preferir STRING.
Pérdida de precisión	Uso de double	Usar BigDecimal.
hashCode inestable	Depende de campos mutables	Usar estrategia estable.
Problemas con Lombok @Data	Incluye relaciones en métodos generados	Usar anotaciones selectivas.

Paso 13 - Reto resuelto — Añadir un campo @Lob binario

Reto: Añadir un campo foto a la entidad Alumno que almacene una imagen como byte[] mapeado a BLOB.

Solución paso a paso:

Subpaso 1. Añadir el campo a la entidad:

```
@Lob
@Column(name = "foto")
private byte[] foto;

public byte[] getFoto() { return foto; }
public void setFoto(byte[] foto) { this.foto = foto; }
```

Subpaso 2. Arrancar y ver el SQL:

```
Hibernate: create table alumnos (
    ...
    foto blob,
    ...
)
```

El campo se mapea a BLOB.

Subpaso 3. Añadir un endpoint para subir la foto:

```
@PostMapping("/{id}/foto")
public ResponseEntity<Void> subirFoto(
    @PathVariable Long id,
```

```

    @RequestParam("fichero") MultipartFile fichero) throws IOException {
        return service.subirFoto(id, fichero.getBytes())
            ? ResponseEntity.noContent().build()
            : ResponseEntity.notFound().build();
    }
}

```

Subpaso 4. Añadir el método al servicio:

```

@Transactional
public boolean subirFoto(Long id, byte[] foto) {
    return repositorio.findById(id).map(entidad -> {
        entidad.setFoto(foto);
        repositorio.save(entidad);
        return true;
    }).orElse(false);
}

```

Subpaso 5. Probar con curl:

```

curl -i -X POST http://localhost:8080/api/v1/alumnos/1/foto \
-F "fichero=@imagen.jpg"

```

Pregunta de reflexión: ¿Qué diferencia hay entre @Lob con String y @Lob con byte[]?

Resultado esperado global

Al final del ejercicio, deberías tener:

- Una entidad Alumno con:
 - Campos básicos (nombre, apellidos, dni, fechaNacimiento, curso).
 - Un enum EstadoAlumno con @Enumerated(STRING).
 - Un campo @Transient (edad).
 - Una dirección embebida (@Embedded).
 - Un campo @Version para bloqueo optimista.
 - Un campo @Lob para observaciones.
- Un DTO actualizado con los nuevos campos.
- Un servicio que transforma entre entidad y DTO.
- equals y hashCode implementados correctamente.

Resumen técnico

El ejercicio ha demostrado:

- Ciclo de vida de una entidad: transient, managed, detached, removed.
- Generación de IDs: IDENTITY.
- @Column: name, nullable, unique, length.
- @Enumerated(EnumType.STRING): para enums.
- @Transient: campos no persistidos.
- @Embedded y @Embeddable: grupos de campos reutilizables.
- @Version: bloqueo optimista.
- @Lob: para objetos grandes.
- equals y hashCode: basados en el ID.
- Transformación DTO ↔ Entidad.

Conclusión

En este punto 4.2 hemos profundizado en la definición de entidades JPA:

- Ciclo de vida: transient, managed, detached, removed.
- Requisitos: @Entity, @Id, constructor sin argumentos, no final.
- Generación de IDs: IDENTITY, SEQUENCE, AUTO, TABLE.
- Mapeo de columnas: @Column, tipos de datos, BigDecimal, LocalDate.
- Tipos especiales: @Enumerated, @Lob, @Temporal.
- Anotaciones avanzadas: @Transient, @Embedded, @Version.
- Buenas prácticas: DTOs, wrapper types, EnumType.STRING, equals y hashCode.
- Errores comunes: constructor, final, Double, @Data. En la práctica, hemos ampliado la entidad Alumno con campos de distintos tipos, un enum, un campo calculado, una dirección embebida y un campo de versión. Hemos visto cómo Hibernate genera el SQL y cómo se comporta la entidad en la base de datos.

La idea clave: una entidad bien definida es la base de una persistencia sólida. Elegir bien los tipos, las anotaciones y las estrategias de generación de IDs evita problemas futuros.

En el siguiente punto, 4.3 - Repositorios JPA, profundizaremos en los repositorios de Spring Data JPA: métodos derivados, @Query, paginación, ordenación, y consultas personalizadas.

Fin del Punto 4.2.

Práctica 4.3 - Repositorios JPA

Contexto del ejercicio: Vamos a ampliar el AlumnoRepository con métodos derivados, consultas con @Query, paginación y ordenación. Veremos cómo se comporta cada uno y cómo se integran en el servicio.

Requisitos previos: Tener el proyecto mi-proyecto con la entidad Alumno del punto 4.2 y el repositorio básico.

Paso 1 - Repasar el repositorio actual

Abre AlumnoRepository y observa su contenido:

```
public interface AlumnoRepository extends JpaRepository<Alumno, Long> {  
  
    Optional<Alumno> findByDni(String dni);  
  
    boolean existsByDni(String dni);  
}
```

Solo tiene dos métodos derivados. Vamos a ampliarlo.

Pregunta de reflexión: ¿Qué otros métodos crees que son útiles en un repositorio de alumnos?

Paso 2 - Añadir métodos derivados

Vamos a añadir varios métodos derivados:

```
public interface AlumnoRepository extends JpaRepository<Alumno, Long> {  
  
    Optional<Alumno> findByDni(String dni);  
  
    boolean existsByDni(String dni);  
  
    List<Alumno> findByCurso(String curso);  
  
    List<Alumno> findByEstado(EstadoAlumno estado);  
}
```

```

List<Alumno> findByCursoAndEstado(String curso, EstadoAlumno estado);

List<Alumno> findByFechaNacimientoBetween(LocalDate desde, LocalDate hasta);

List<Alumno> findByNombreContainingIgnoreCase(String texto);

List<Alumno> findByCursoOrderByApellidosAsc(String curso);

long countByCurso(String curso);

void deleteByDni(String dni);
}

```

Cada método tiene su consulta generada: - findByCurso → SELECT a FROM Alumno a WHERE a.curso = ?. - findByEstado → SELECT a FROM Alumno a WHERE a.estado = ?. - findByCursoAndEstado → SELECT a FROM Alumno a WHERE a.curso = ? AND a.estado = ?. - findByFechaNacimientoBetween → SELECT a FROM Alumno a WHERE a.fechaNacimiento BETWEEN ? AND ?. - findByNombreContainingIgnoreCase → SELECT a FROM Alumno a WHERE LOWER(a.nombre) LIKE LOWER(CONCAT('%', ?, '%')). - findByCursoOrderByApellidosAsc → SELECT a FROM Alumno a WHERE a.curso = ? ORDER BY a.apellidos ASC. - countByCurso → SELECT COUNT(a) FROM Alumno a WHERE a.curso = ?. - deleteByDni → DELETE FROM Alumno a WHERE a.dni = ?.

Pregunta de reflexión: ¿Qué diferencia hay entre findByCurso y findByCursoOrderByApellidosAsc?

Paso 3 - Arrancar y verificar

Reinicia la aplicación. En los logs verás que Spring Data JPA valida los métodos al arrancar. Si algún nombre no se puede interpretar, verás un error.

```

...
Started MiProyectoApplication in X.XXX seconds

```

Si todo va bien, los métodos están listos para usarse.

Pregunta de reflexión: ¿Qué pasaría si escribieras findByCursoo (con dos “o”)? ¿Cuándo te enterarías del error?

Paso 4 - Añadir un método con @Query

Vamos a añadir un método que busque alumnos de un curso nacidos después de una fecha:

```

@Query(
    "SELECT a FROM Alumno a "
    + "WHERE a.curso = :curso AND a.fechaNacimiento > :fecha"
)
List<Alumno> buscarPorCursoYNacidosDespues(
    @Param("curso") String curso,
    @Param("fecha") LocalDate fecha);

```

@Query permite declarar manualmente la consulta JPQL.

SELECT a FROM Alumno a es JPQL: se usa el nombre de la clase, no de la tabla.

:curso y :fecha son parámetros nombrados. Se referencian con @Param.

Pregunta de reflexión: ¿Qué diferencia hay entre este método y findByCursoAndFechaNacimientoAfter (derivado)?

Paso 5 - Añadir un método con @Modifying

Vamos a añadir un método que actualice el estado de todos los alumnos de un curso:

```
@Modifying
@Query("UPDATE Alumno a SET a.estado = :estado WHERE a.curso = :curso")
int actualizarEstadoPorCurso(
    @Param("curso") String curso,
    @Param("estado") EstadoAlumno estado);
```

@Modifying indica que la consulta realiza una escritura.

El valor de retorno es int, con el número de filas afectadas.

Importante: este método debe llamarse desde un método del servicio anotado con @Transactional. Si no, Spring lanza TransactionRequiredException.

Pregunta de reflexión: ¿Por qué es importante que el método sea transaccional?

Paso 6 - Usar el método @Modifying desde el servicio

Añade el método al servicio:

```
@Transactional
public int actualizarEstadoPorCurso(String curso, EstadoAlumno estado) {
    return repositorio.actualizarEstadoPorCurso(curso, estado);
}
```

El servicio delimita la transacción necesaria para ejecutar la consulta @Modifying.

Y añade el endpoint al controlador:

```
@PatchMapping("/curso/{curso}/estado")
public ResponseEntity<Map<String, Object>> actualizarEstado(
    @PathVariable String curso,
    @RequestParam EstadoAlumno estado) {
    int actualizados = service.actualizarEstadoPorCurso(curso, estado);
    return ResponseEntity.ok(Map.of(
        "actualizados", actualizados,
        "curso", curso,
        "estado", estado));
}
```

Pregunta de reflexión: ¿Qué pasa si no se anota el método del servicio con @Transactional?

Paso 7 - Añadir paginación

Vamos a añadir un método que devuelva una página de alumnos:

En el repositorio, JpaRepository ya tiene findAll(Pageable). No hace falta declararlo. Pero podemos añadir uno con filtro:

```
Page<Alumno> findByCurso(String curso, Pageable pageable);
```

En el servicio:

```
@Transactional(readOnly = true)
public Page<AlumnoResponseDTO> listarPaginado(int page, int size, String sort) {
    Pageable pageable = PageRequest.of(page, size, Sort.by(sort).ascending());
    return repositorio.findAll(pageable).map(this::toDTO);
}
```

PageRequest.of(page, size, sort) construye el Pageable. repositorio.findAll(pageable) devuelve Page.

.map(this::toDTO) transforma cada entidad a DTO. El resultado es Page.

En el controlador:

```
@GetMapping("/paginado")
public Page<AlumnoResponseDTO> listarPaginado(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size,
    @RequestParam(defaultValue = "id") String sort) {
    return service.listarPaginado(page, size, sort);
}
```

Pregunta de reflexión: ¿Qué metadatos incluye Page en la respuesta JSON?

Paso 8 - Probar la paginación

Reinicia y prueba:

```
# Página 0, tamaño 2, ordenado por nombre
curl "http://localhost:8080/api/v1/alumnos/paginado" \
  --get --data "page=0" --data "size=2" --data "sort=nombre"

# Página 1, tamaño 2
curl "http://localhost:8080/api/v1/alumnos/paginado" \
  --get --data "page=1" --data "size=2" --data "sort=nombre"
```

Verás una respuesta como:

```
{
  "content": [
    {"id": "1", "nombre": "Ana", ...},
    {"id": "2", "nombre": "Luis", ...}
  ],
  "pageable": {...},
  "totalElements": 3,
  "totalPages": 2,
  "number": 0,
  "size": 2,
  "first": true,
  "last": false
}
```

Pregunta de reflexión: ¿Qué diferencia hay entre totalElements y size?

Paso 9 - Añadir Specifications

Vamos a permitir que el repositorio use Specifications. Modifica la interfaz:

```
public interface AlumnoRepository extends JpaRepository<Alumno, Long>,
    JpaSpecificationExecutor<Alumno> {
    // ...
}
```

JpaSpecificationExecutor añade métodos que aceptan Specification:

- findAll(Specification).
- findAll(Specification, Pageable).
- count(Specification).

- exists(Specification).

Pregunta de reflexión: ¿Qué ventaja tiene que el repositorio extienda JpaSpecificationExecutor?

Paso 10 - Crear una clase de Specifications

Crea la clase AlumnoSpecifications:

```
package es.mecd.demo.miproyecto.alumno;

import org.springframework.data.jpa.domain.Specification;
import java.time.LocalDate;

public class AlumnoSpecifications {

    public static Specification<Alumno> porCurso(String curso) {
        return (root, query, cb) ->
            curso == null || curso.isBlank()
                ? null
                : cb.equal(root.get("curso"), curso);
    }

    public static Specification<Alumno> porEstado(EstadoAlumno estado) {
        return (root, query, cb) ->
            estado == null
                ? null
                : cb.equal(root.get("estado"), estado);
    }

    public static Specification<Alumno> nacidoDespues(LocalDate fecha) {
        return (root, query, cb) ->
            fecha == null
                ? null
                : cb.greaterThan(root.get("fechaNacimiento"), fecha);
    }
}
```

Cada Specification devuelve un Predicate o null. Si devuelve null, se ignora.

(root, query, cb) son los parámetros de la lambda:

- root: el punto de entrada a la entidad (Alumno).
- query: la consulta completa.
- cb: el CriteriaBuilder, que construye los predicados.

cb.equal(root.get("curso"), curso) construye WHERE curso = ?.

cb.greaterThan(root.get("fechaNacimiento"), fecha) construye WHERE fechaNacimiento > ?.

Pregunta de reflexión: ¿Por qué las Specifications devuelven null cuando el filtro es nulo? ¿Qué pasaría si no lo hicieran?

Paso 11 - Usar Specifications en el servicio

Añade el método al servicio:

```
@Transactional(readOnly = true)
public List<AlumnoResponseDTO> buscarConFiltros(
    String curso,
    EstadoAlumno estado,
```

```

    LocalDate nacidoDespues) {
    Specification<Alumno> spec = Specification
        .where(AlumnoSpecifications.porCurso(curso))
        .and(AlumnoSpecifications.porEstado(estado))
        .and(AlumnoSpecifications.nacidoDespues(nacidoDespues));
    return repositorio.findAll(spec).stream()
        .map(this::toDTO)
        .toList();
}

```

Specification.where(...) empieza la construcción.

.and(...) combina Specifications. Se pueden añadir tantas como filtros haya.

repositorio.findAll(spec) ejecuta la consulta combinada.

Y el endpoint en el controlador:

```

@GetMapping("/buscar")
public List<AlumnoResponseDTO> buscar(
    @RequestParam(required = false) String curso,
    @RequestParam(required = false) EstadoAlumno estado,
    @RequestParam(required = false)
    @DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
    LocalDate nacidoDespues) {
    return service.buscarConFiltros(curso, estado, nacidoDespues);
}

```

Pregunta de reflexión: ¿Qué pasa si el cliente no envía ningún filtro? ¿Qué devuelve la consulta?

Paso 12 - Probar Specifications

Reinicia y prueba:

```

# Sin filtros (devuelve todos)
curl "http://localhost:8080/api/v1/alumnos/buscar"

# Solo curso
curl "http://localhost:8080/api/v1/alumnos/buscar?curso=5%C2%BA%20Primaria"

# Curso y estado
curl "http://localhost:8080/api/v1/alumnos/buscar" \
  --get \
  --data-urlencode "curso=5º Primaria" \
  --data-urlencode "estado=ACTIVO"

# Curso, estado y fecha
curl "http://localhost:8080/api/v1/alumnos/buscar" \
  --get \
  --data-urlencode "curso=5º Primaria" \
  --data-urlencode "estado=ACTIVO" \
  --data-urlencode "nacidoDespues=2010-01-01"

```

Cada combinación devuelve los alumnos que cumplen los filtros enviados.

Pregunta de reflexión: ¿Qué ventaja tiene este enfoque frente a tener un método derivado por cada combinación de filtros?

Paso 13 - Errores comunes del ejercicio

Error	Causa	Solución
PropertyReferenceException	Campo mal escrito en método derivado	Corregir según la entidad.
TransactionRequiredException	@Modifying sin transacción	Usar servicio @Transactional.
InvalidDataAccessApiUsageException	Falta de JpaSpecificationExecutor	Añadir la interfaz.
Página vacía	Índice/tamaño incorrecto	Primera página = 0; validar límites.
Sort inválido	Campo no existe	Limitar campos permitidos.
NPE en Specification	Filtro opcional no gestionado	Usar predicado neutro.
Consulta nativa no portable	SQL específico	Preferir JPQL/Criteria.
LazyInitializationException	Relación usada tarde	DTO/fetch explícito.

Paso 14 - Reto resuelto — Consulta con agregación

Reto: Añadir un método al repositorio que devuelva el número de alumnos por curso.

Solución paso a paso: **Subpaso 1.** Añadir el método al repositorio:

```
@Query("SELECT a.curso, COUNT(a) FROM Alumno a GROUP BY a.curso")
List<Object[]> contarPorCurso();
```

Subpaso 2. Añadir el método al servicio:

```
@Transactional(readOnly = true)
public Map<String, Long> contarPorCurso() {
    return repositorio.contarPorCurso().stream()
        .collect(Collectors.toMap(
            fila -> (String) fila[0],
            fila -> (Long) fila[1]
        ));
}
```

Subpaso 3. Añadir el endpoint al controlador:

```
@GetMapping("/estadisticas/por-curso")
public Map<String, Long> contarPorCurso() {
    return service.contarPorCurso();
}
```

Subpaso 4. Probar:

```
curl http://localhost:8080/api/v1/alumnos/estadisticas/por-curso
```

Devuelve algo como:

```
{
  "5º Primaria": 1,
  "6º Primaria": 1,
  "4º Primaria": 1
}
```

Pregunta de reflexión: ¿Qué diferencia hay entre una consulta de agregación y una consulta normal?

Resultado esperado global

Al final del ejercicio, deberías tener:

- Un AlumnoRepository con:
 - Métodos derivados (findByCurso, findByEstado, etc.).
 - Métodos con @Query (buscarPorCursoYNacidosDespues).
 - Métodos con @Modifying (actualizarEstadoPorCurso).
 - Paginación (findByCurso(String, Pageable)).
 - Specifications (JpaSpecificationExecutor).
- Un AlumnoService con @Transactional en los métodos que lo requieren.
- Un AlumnoController con endpoints que usan estos métodos.
- Una clase AlumnoSpecifications con filtros combinables.

Resumen técnico

El ejercicio ha demostrado:

- Jerarquía de repositorios: JpaRepository como base.
- Métodos derivados: findBy, existsBy, countBy, deleteBy.
- @Query: JPQL y SQL nativo.
- @Modifying: actualizaciones y borrados masivos.
- Paginación: Pageable, Page, Sort.
- Specifications: consultas dinámicas con JpaSpecificationExecutor.
- Agregaciones: GROUP BY con @Query.
- Buenas prácticas: limitar size, ordenación por defecto, documentar.

Conclusión

En este punto 4.3 hemos profundizado en los repositorios JPA:

- Jerarquía: Repository → CrudRepository → PagingAndSortingRepository → JpaRepository.
- Métodos derivados: Spring genera la consulta a partir del nombre.
- @Query: JPQL o SQL nativo para consultas personalizadas.
- @Modifying: actualizaciones y borrados masivos.
- Paginación y ordenación: Pageable, Page, Sort.
- Specifications: consultas dinámicas con filtros opcionales.
- Agregaciones: GROUP BY con @Query.

En la práctica, hemos ampliado el repositorio con métodos derivados, consultas personalizadas, paginación, Specifications y agregaciones. Hemos visto cómo cada enfoque se adapta a un tipo de consulta. La idea clave: Spring Data JPA elimina la mayor parte del trabajo de acceso a datos. Tú defines el método; Spring genera la consulta. Para consultas complejas, @Query y Specifications son las herramientas.

En el siguiente punto, 4.4 - Relaciones entre entidades: OneToMany, ManyToOne, profundizaremos en las relaciones entre entidades: cómo modelarlas, cómo se mapean a la base de datos, cómo se cargan y cómo se evitan los problemas de rendimiento.

Fin del Punto 4.3.

Práctica 4.4 - Relaciones entre entidades: OneToMany, ManyToOne

Contexto del ejercicio: Vamos a modelar una relación @OneToMany / @ManyToOne entre Curso y Alumno. Un curso tiene muchos alumnos; un alumno pertenece a un curso. Veremos cómo se mapea a la base de datos, cómo se mantiene la relación con métodos helper, y cómo se evita el problema N+1.

Requisitos previos: Tener el proyecto mi-proyecto con la entidad Alumno del punto 4.3.

Paso 1 - Crear la entidad Curso

Crea la clase Curso en el paquete curso (o en alumno, si prefieres mantenerlo junto):

```
package es.mecd.demo.miproyecto.curso;

import es.mecd.demo.miproyecto.alumno.Alumno;
import jakarta.persistence.*;

import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "cursos")
public class Curso {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String nombre;

    @OneToMany(
        mappedBy = "curso",
        cascade = CascadeType.ALL,
        orphanRemoval = true,
        fetch = FetchType.LAZY)
    private List<Alumno> alumnos = new ArrayList<>();

    public Curso() {
    }

    public Curso(String nombre) {
        this.nombre = nombre;
    }

    public void addAlumno(Alumno alumno) {
        alumnos.add(alumno);
        alumno.setCurso(this);
    }

    public void removeAlumno(Alumno alumno) {
        alumnos.remove(alumno);
        alumno.setCurso(null);
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public List<Alumno> getAlumnos() { return alumnos; }
    public void setAlumnos(List<Alumno> alumnos) { this.alumnos = alumnos; }
}
```

Detalles clave:

- `@OneToMany(mappedBy = "curso")`: el lado inverso. `mappedBy` apunta al campo `curso` de `Alumno`.
- `cascade = CascadeType.ALL`: al guardar o eliminar un curso, se propaga a los alumnos.
- `orphanRemoval = true`: si se quita un alumno de la lista, se elimina de la base de datos.
- `fetch = FetchType.LAZY`: los alumnos no se cargan hasta que se accede a la lista.
- `addAlumno` y `removeAlumno`: métodos helper que mantienen ambos lados de la relación.

Pregunta de reflexión: ¿Qué pasaría si no se pone `mappedBy = "curso"`?

Paso 2 - Modificar la entidad `Alumno` para añadir la relación

Añade el campo `curso` a la entidad `Alumno`:

```
@Entity
@Table(name = "alumnos")
public class Alumno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // ... otros campos
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "curso_id", nullable = false)
    private Curso curso;

    public Curso getCurso() { return curso; }
    public void setCurso(Curso curso) { this.curso = curso; }

    // ... resto de getters y setters
}
```

Detalles:

- `@ManyToOne(fetch = FetchType.LAZY)`: el lado propietario. La relación se carga de forma perezosa.
- `@JoinColumn(name = "curso_id", nullable = false)`: la columna de la clave foránea. Obligatoria.

Migración necesaria desde 4.3: desde este momento `curso` deja de ser `String` y pasa a ser una entidad `Curso`. Los métodos derivados que antes usaban `findByCurso(...)` deben navegar por la propiedad relacionada, por ejemplo `findByCursoNombreIgnoreCaseAndEliminadoFalse(...)` o `findByCursoIdAndEliminadoFalse(...)`. Las consultas JPQL deben usar `a.curso.nombre` cuando filtran por el nombre. Esta adaptación evita que Spring Data falle al arrancar con `PropertyReferenceException`.

Pregunta de reflexión: ¿Por qué el `@ManyToOne` es el lado propietario y no el `@OneToMany`?

Paso 3 - Arrancar y ver el SQL generado

Reinicia la aplicación. En los logs verás el SQL de creación de las tablas y de la clave foránea:

```
Hibernate: create table cursos (
  id bigint generated by default as identity,
  nombre varchar(50) not null,
  primary key (id)
)
Hibernate: create table alumnos (
  ...
```

```

        curso_id bigint not null,
        ...
        primary key (id)
    )
Hibernate: alter table if exists alumnos
    add constraint FK...
    foreign key (curso_id) references cursos

```

Observa:

- La tabla cursos se crea con su ID y nombre.
- La tabla alumnos tiene una nueva columna curso_id que apunta a cursos.
- Se añade una restricción de clave foránea.
- No se crea tabla intermedia. mappedBy ha funcionado.

Pregunta de reflexión: ¿Qué pasaría si no hubiera mappedBy en el lado @OneToMany?

Paso 4 - Crear el repositorio de Curso

Crea la interfaz CursoRepository:

```

package es.mecd.demo.miproyecto.curso;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface CursoRepository extends JpaRepository<Curso, Long> {

    Optional<Curso> findByNombre(String nombre);

    boolean existsByNombre(String nombre);
}

```

Pregunta de reflexión: ¿Qué métodos crees que son útiles en un repositorio de cursos?

Paso 5 - Crear el DTO de Curso

Crea CursoDTO:

```

package es.mecd.demo.miproyecto.curso;

import com.fasterxml.jackson.annotation.JsonInclude;
import com.fasterxml.jackson.annotation.JsonProperty;

@JsonInclude(JsonInclude.Include.NON_NULL)
public class CursoDTO {

    @JsonProperty("id")
    private String identificador;

    private String nombre;

    private Integer numeroAlumnos;

    public CursoDTO() {
    }

    public String getIdentificador() { return identificador; }
}

```

```

    public void setIdentificador(String identificador) {
        this.identificador = identificador;
    }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public Integer getNumeroAlumnos() { return numeroAlumnos; }
    public void setNumeroAlumnos(Integer numeroAlumnos) {
        this.numeroAlumnos = numeroAlumnos;
    }
}

```

El campo numeroAlumnos es un campo calculado: no está en la entidad, se calcula al transformar.

Pregunta de reflexión: ¿Por qué el campo numeroAlumnos no está en la entidad?

Paso 6 - Crear el servicio de Curso

Crea CursoService:

```

package es.mecd.demo.miproyecto.curso;

import es.mecd.demo.miproyecto.common.exception.NegocioException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.Optional;

@Service
public class CursoService {

    private final CursoRepository repositorio;

    public CursoService(CursoRepository repositorio) {
        this.repositorio = repositorio;
    }

    @Transactional(readOnly = true)
    public List<CursoDTO> listar() {
        return repositorio.findAll().stream()
            .map(this::toDTO)
            .toList();
    }

    @Transactional(readOnly = true)
    public Optional<CursoDTO> consultar(Long id) {
        return repositorio.findById(id).map(this::toDTO);
    }

    @Transactional
    public CursoDTO crear(String nombre) {
        if (repositorio.existsByNombre(nombre)) {
            throw new NegocioException("Ya existe un curso con el nombre " + nombre);
        }
        Curso curso = new Curso(nombre);
        return toDTO(repositorio.save(curso));
    }
}

```

```

    }

    private CursoDTO toDTO(Curso curso) {
        CursoDTO dto = new CursoDTO();
        dto.setIdentificador(
            curso.getId() != null
                ? curso.getId().toString()
                : null);
        dto.setNombre(curso.getNombre());
        dto.setNumeroAlumnos(curso.getAlumnos().size());
        return dto;
    }
}

```

`curso.getAlumnos().size()` accede a una colección LAZY. Debe hacerse dentro de una transacción activa o, preferiblemente, mediante una consulta que cargue exactamente los datos requeridos para el DTO.

Pregunta de reflexión: ¿Por qué es importante que `toDTO` se llame dentro de la transacción?

Paso 7 - Modificar el `AlumnoService` para asignar el curso

El método `crear` del `AlumnoService` debe buscar el curso y asignarlo al alumno. Modifica el método:

```

@Transactional
public AlumnoResponseDTO crear(AlumnoRequestDTO request) {
    if (repositorio.existsByDni(request.getDni())) {
        throw new NegocioException(
            "Ya existe un alumno con el DNI " + request.getDni());
    }
    Curso curso = cursoRepository.findByNombre(request.getCurso())
        .orElseThrow(() -> new NegocioException(
            "Curso no encontrado: " + request.getCurso()));
    Alumno entidad = new Alumno(
        request.getNombre(),
        request.getApellidos(),
        request.getDni(),
        request.getFechaNacimiento()
    );
    entidad.setCurso(curso);
    Alumno guardado = repositorio.save(entidad);
    return toDTO(guardado);
}

```

Detalles: - `cursoRepository.findByNombre(...)` busca el curso por nombre. Si no existe, lanza `NegocioException`. - `entidad.setCurso(curso)` establece la relación en el lado propietario. Es lo que se persiste en la base de datos. - No se usa `curso.addAlumno(entidad)` aquí. El método `helper` se usa cuando se gestiona la relación desde el lado del curso. En este caso, estamos creando el alumno y asignándole un curso, así que basta con establecer el curso en el alumno.

Pregunta de reflexión: ¿Por qué no basta con hacer `curso.addAlumno(entidad)` sin `entidad.setCurso(curso)`?

Paso 8 - Crear el controlador de Curso

Crea `CursoController`:

```

package es.mecd.demo.miproyecto.curso;

import es.mecd.demo.miproyecto.alumno.AlumnoResponseDTO;
import es.mecd.demo.miproyecto.alumno.AlumnoService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.net.URI;
import java.util.List;

@RestController
@RequestMapping("/api/v1/cursos")
public class CursoController {

    private final CursoService service;
    private final AlumnoService alumnoService;

    public CursoController(CursoService service, AlumnoService alumnoService) {
        this.service = service;
        this.alumnoService = alumnoService;
    }

    @GetMapping
    public List<CursoDTO> listar() {
        return service.listar();
    }

    @GetMapping("/{id}")
    public ResponseEntity<CursoDTO> consultar(@PathVariable Long id) {
        return service.consultar(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<CursoDTO> crear(@RequestParam String nombre) {
        CursoDTO creado = service.crear(nombre);
        URI location = URI.create("/api/v1/cursos/" + creado.getIdentificador());
        return ResponseEntity.created(location).body(creado);
    }

    @GetMapping("/{id}/alumnos")
    public List<AlumnoResponseDTO> listarAlumnos(@PathVariable Long id) {
        return alumnoService.listarPorCurso(id);
    }
}

```

listarAlumnos devuelve los alumnos de un curso. Delega en el AlumnoService.

Pregunta de reflexión: ¿Por qué el controlador de Curso inyecta el AlumnoService?

Paso 9 - Añadir el método listarPorCurso al AlumnoService

```

@Transactional(readOnly = true)
public List<AlumnoResponseDTO> listarPorCurso(Long cursoId) {
    return repositorio.findByCursoId(cursoId).stream()
        .map(this::toDTO)
}

```

```
        .toList();
    }
}
```

Y añade el método al repositorio:

```
List<Alumno> findByCursoId(Long cursoId);
```

findByCursoId busca los alumnos cuyo curso tiene ese ID. Spring Data JPA genera la consulta a partir del nombre.

Pregunta de reflexión: ¿Qué consulta SQL genera findByCursoId?

Paso 10 - Arrancar y probar

Reinicia la aplicación. Prueba a crear un curso y un alumno:

```
# Crear un curso
curl -i -X POST "http://localhost:8080/api/v1/cursos?nombre=5%C2%BA%20Primaria"

# Crear un alumno en ese curso
curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{
    "nombre": "Ana",
    "apellidos": "García López",
    "dni": "12345678A",
    "fechaNacimiento": "2010-05-12",
    "curso": "5º Primaria"
  }'

# Listar alumnos del curso
curl http://localhost:8080/api/v1/cursos/1/alumnos

# Consultar el curso (verás el número de alumnos)
curl http://localhost:8080/api/v1/cursos/1
```

Pregunta de reflexión: ¿Qué devuelve el campo numeroAlumnos del curso?

Paso 11 - Detectar y resolver el problema N+1

Vamos a provocar el problema N+1 para verlo en los logs. Modifica el método listar del CursoService para que acceda a los alumnos de cada curso:

```
@Transactional(readOnly = true)
public List<CursoDTO> listar() {
    return repositorio.findAll().stream()
        .map(this::toDTO)
        .toList();
}
```

Si tienes varios cursos, verás en los logs:

```
Hibernate: select c.* from cursos c
Hibernate: select a.* from alumnos a where a.curso_id = ?
Hibernate: select a.* from alumnos a where a.curso_id = ?
Hibernate: select a.* from alumnos a where a.curso_id = ?
```

Una consulta por cada curso. Es el problema N+1.

Para solucionarlo, añade un método al repositorio con JOIN FETCH:

```
@Query("SELECT DISTINCT c FROM Curso c LEFT JOIN FETCH c.alumnos")
List<Curso> findAllConAlumnos();
```

Y modifica el servicio para usarlo:

```
@Transactional(readOnly = true)
public List<CursoDTO> listar() {
    return repositorio.findAllConAlumnos().stream()
        .map(this::toDTO)
        .toList();
}
```

Ahora verás una sola consulta con JOIN:

Hibernate: select c.*, a.* from cursos c left join alumnos a on a.curso_id = c.id

Pregunta de reflexión: ¿Por qué LEFT JOIN FETCH en lugar de JOIN FETCH? ¿Qué diferencia hay?

Paso 12 - Errores comunes del ejercicio

Error	Causa	Solución
LazyInitializationException	Relación LAZY fuera de transacción	DTO/fetch específico.
Tabla intermedia innecesaria	Falta mappedBy	Definir lado propietario.
DataIntegrityViolationException	FK a curso inválido	Validar curso.
StackOverflowError	Relación bidireccional serializada	Usar DTOs.
NPE en colección	Colección no inicializada	new ArrayList<>().
N+1 consultas	LAZY recorrido en bucle	JOIN FETCH/@EntityGraph.
Eliminación en cascada excesiva	Cascade demasiado amplio	Usar sólo cascades necesarios.
orphanRemoval no actúa	Relación desincronizada	Usar helpers add/remove.

Paso 13 - Reto resuelto — Eliminar un curso con cascade

Reto: Eliminar un curso. Como tiene cascade = CascadeType.ALL y orphanRemoval = true, al eliminar el curso se deben eliminar también sus alumnos.

Solución paso a paso:

Subpaso 1. Añadir el método al servicio:

```
@Transactional
public boolean eliminar(Long id) {
    if (!repositorio.existsById(id)) {
        return false;
    }
    repositorio.deleteById(id);
    return true;
}
```

Subpaso 2. Añadir el endpoint al controlador:

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable Long id) {
    return service.eliminar(id)
        ? ResponseEntity.noContent().build()
        : ResponseEntity.notFound().build();
}
```

Subpaso 3. Probar:

```
curl -i -X DELETE http://localhost:8080/api/v1/cursos/1
```

Verás en los logs:

Hibernate: delete from alumnos where id=?

Hibernate: delete from alumnos where id=?

Hibernate: delete from cursos where id=?

Hibernate elimina primero los alumnos (por el cascade) y luego el curso. > **Pregunta de reflexión:** ¿Qué pasaría si el curso tuviera 1000 alumnos? ¿Sería eficiente?

Resultado esperado global

Al final del ejercicio, deberías tener:

- Una entidad Curso con @OneToMany hacia Alumno.
- Una entidad Alumno con @ManyToOne hacia Curso.
- Métodos helper addAlumno y removeAlumno en Curso.
- Un CursoRepository y un CursoService.
- Un CursoController con endpoints.
- La relación mapeada a la base de datos con una clave foránea.
- El problema N+1 detectado y resuelto con JOIN FETCH.

Resumen técnico

El ejercicio ha demostrado:

- @ManyToOne: lado propietario, con @JoinColumn. FetchType.LAZY.
- @OneToMany: lado inverso, con mappedBy. FetchType.LAZY.
- mappedBy: evita tabla intermedia.
- Métodos helper: addAlumno, removeAlumno.
- cascade y orphanRemoval.
- Problema N+1: detección y solución con JOIN FETCH.
- LazyInitializationException: solución transformando a DTO en el servicio.
- DTOs: nunca exponer entidades.

Conclusión

En este punto 4.4 hemos profundizado en las relaciones entre entidades:

- Cuatro tipos: @OneToOne, @OneToMany, @ManyToOne, @ManyToMany.
- Lado propietario: @ManyToOne. Lado inverso: @OneToMany con mappedBy.
- FetchType: LAZY recomendado.
- @JoinColumn: personaliza la clave foránea.
- Métodos helper: addAlumno, removeAlumno.
- cascade y orphanRemoval: composición, agregación, referencia.
- Problema N+1: 1 + N consultas. JOIN FETCH o @EntityGraph.
- LazyInitializationException: acceso a relación LAZY fuera de transacción.

En la práctica, hemos modelado una relación @OneToMany / @ManyToOne entre Curso y Alumno, hemos creado los repositorios y servicios, hemos probado la relación, y hemos detectado y resuelto el problema N+1.

La idea clave: las relaciones son el corazón de una base de datos relacional, pero también su mayor fuente de problemas. Elegir bien el FetchType, usar mappedBy, y transformar a DTO dentro de la transacción evita la mayoría de los errores.

En el siguiente punto, 4.5 - Consultas básicas, profundizaremos en las consultas más comunes con JPA: consultas derivadas, @Query, JOIN FETCH, agregaciones, y consultas con múltiples condiciones.

Fin del Punto 4.4.

Práctica 4.5 - Consultas básicas

Contexto del ejercicio: Vamos a ampliar el AlumnoRepository y el CursoRepository con consultas JPQL de distintos tipos: joins, funciones, agregaciones y proyecciones. Veremos cómo se comporta cada una y cómo se integran en el servicio.

Requisitos previos: Tener el proyecto mi-proyecto con las entidades Alumno y Curso del punto 4.4.

Paso 1 - Repasar el repositorio actual

Abre AlumnoRepository y observa su contenido. Tiene métodos derivados y algunos @Query del punto 4.3. Vamos a ampliarlo con consultas más avanzadas.

Pregunta de reflexión: ¿Qué consultas crees que son útiles en un repositorio de alumnos con relación a cursos?

Paso 2 - Consulta con JOIN simple

Vamos a añadir un método que busque alumnos por nombre de curso, usando JOIN:

```
@Query("SELECT a FROM Alumno a JOIN a.curso c WHERE c.nombre = :nombre")
List<Alumno> buscarPorNombreCurso(@Param("nombre") String nombre);
```

JOIN a.curso c une la entidad Alumno con su Curso. El alias c permite referenciar el curso en la cláusula WHERE.

c.nombre = :nombre filtra por el nombre del curso.

Esta consulta devuelve los alumnos. El curso no se carga en la consulta (no hay FETCH).

Pregunta de reflexión: ¿Qué pasa si accedes a alumno.getCurso() después de esta consulta?

Paso 3 - Consulta con JOIN FETCH

Ahora añade un método con JOIN FETCH para cargar el curso:

```
@Query("SELECT a FROM Alumno a JOIN FETCH a.curso WHERE a.curso.nombre = :nombre")
List<Alumno> buscarPorNombreCursoConCurso(@Param("nombre") String nombre);
```

JOIN FETCH a.curso carga el curso en la misma consulta. Cuando accedas a alumno.getCurso(), ya está cargado.

Observa que no hay alias para a.curso: con FETCH, no se puede usar alias. En la cláusula WHERE se referencia directamente a.curso.nombre.

Pregunta de reflexión: ¿Qué diferencia hay entre este método y el anterior? ¿Cuál es más eficiente?

Paso 4 - Consulta con función de string

Vamos a añadir un método que busque por nombre ignorando mayúsculas:

```
@Query("SELECT a FROM Alumno a WHERE LOWER(a.nombre) = LOWER(:nombre)")
List<Alumno> buscarPorNombreIgnoreCase(@Param("nombre") String nombre);
```

LOWER(a.nombre) convierte el nombre del alumno a minúsculas. LOWER(:nombre) hace lo mismo con el parámetro. Así la comparación ignora mayúsculas.

Pregunta de reflexión: ¿Qué consulta SQL genera Hibernate para este método?

Paso 5 - Consulta con función de fecha

Vamos a añadir un método que busque alumnos nacidos en un año concreto:

```
@Query("SELECT a FROM Alumno a WHERE YEAR(a.fechaNacimiento) = :anio")
List<Alumno> buscarPorAnioNacimiento(@Param("anio") int anio);
```

YEAR(a.fechaNacimiento) extrae el año de la fecha de nacimiento.

Pregunta de reflexión: ¿Qué desventaja tiene usar YEAR en lugar de un rango de fechas?
¿Cuándo usarías cada uno?

Paso 6 - Consulta con agregación

Vamos a añadir un método que cuente los alumnos por curso:

```
@Query("SELECT a.curso.nombre, COUNT(a) FROM Alumno a GROUP BY a.curso.nombre")
List<Object[]> contarPorCurso();
```

SELECT a.curso.nombre, COUNT(a) devuelve dos columnas: el nombre del curso y el número de alumnos.

GROUP BY a.curso.nombre agrupa por curso.

El resultado es una lista de arrays Object[], cada uno con dos elementos.

Pregunta de reflexión: ¿Por qué el resultado es List<Object[]> y no List?

Paso 7 - Consulta con HAVING

Vamos a añadir un método que devuelva los cursos con más de N alumnos:

```
@Query("""
    SELECT a.curso.nombre, COUNT(a)
    FROM Alumno a
    GROUP BY a.curso.nombre
    HAVING COUNT(a) > :minimo
""")
List<Object[]> contarCursosConMasDe(@Param("minimo") long minimo);
```

HAVING COUNT(a) > :minimo filtra los grupos que tienen más de :minimo alumnos. > **Pregunta de reflexión:** ¿Qué diferencia hay entre WHERE COUNT(a) > :minimo y HAVING COUNT(a) > :minimo?

Paso 8 - Consulta con proyección a DTO

Vamos a crear un DTO AlumnoResumenDTO y una consulta que lo devuelva directamente:

```
package es.mecd.demo.miproyecto.alumno;
```

```
public class AlumnoResumenDTO {

    private Long id;
    private String nombreCompleto;
    private String curso;

    public AlumnoResumenDTO(Long id, String nombreCompleto, String curso) {
```

```

        this.id = id;
        this.nombreCompleto = nombreCompleto;
        this.curso = curso;
    }

    public Long getId() { return id; }
    public String getNombreCompleto() { return nombreCompleto; }
    public String getCurso() { return curso; }
}

```

Y la consulta en el repositorio:

```

@Query("""
    SELECT new es.mecd.demo.miproyecto.alumno.AlumnoResumenDTO(
        a.id, CONCAT(a.nombre, ' ', a.apellidos), a.curso.nombre)
    FROM Alumno a
    WHERE a.curso.nombre = :curso
""")
List<AlumnoResumenDTO> buscarResumenPorCurso(@Param("curso") String curso);

```

La expresión new + ruta completa del DTO + argumentos del constructor hace que Hibernate cree el DTO directamente, sin cargar la entidad completa.

Pregunta de reflexión: ¿Qué ventaja tiene esta consulta frente a devolver la entidad y transformarla en el servicio?

Paso 9 - Añadir los métodos al servicio

Vamos a añadir los métodos al AlumnoService para exponerlos. Añade estos métodos:

```

@Transactional(readOnly = true)
public List<AlumnoResponseDTO> buscarPorNombreCurso(String nombreCurso) {
    return repositorio.buscarPorNombreCursoConCurso(nombreCurso).stream()
        .map(this::toDTO)
        .toList();
}

@Transactional(readOnly = true)
public List<AlumnoResponseDTO> buscarPorAnioNacimiento(int anio) {
    return repositorio.buscarPorAnioNacimiento(anio).stream()
        .map(this::toDTO)
        .toList();
}

@Transactional(readOnly = true)
public Map<String, Long> contarPorCurso() {
    return repositorio.contarPorCurso().stream()
        .collect(Collectors.toMap(
            fila -> (String) fila[0],
            fila -> (Long) fila[1]
        ));
}

@Transactional(readOnly = true)
public List<AlumnoResumenDTO> buscarResumenPorCurso(String curso) {
    return repositorio.buscarResumenPorCurso(curso);
}

```

Pregunta de reflexión: ¿Por qué contarPorCurso transforma el List<Object[]> a un

Map<String, Long>?

Paso 10 - Añadir los endpoints

Añade los endpoints al AlumnoController:

```
@GetMapping("/por-curso")
public List<AlumnoResponseDTO> buscarPorCurso(@RequestParam String nombre) {
    return service.buscarPorNombreCurso(nombre);
}

@GetMapping("/por-anio")
public List<AlumnoResponseDTO> buscarPorAnio(@RequestParam int anio) {
    return service.buscarPorAnioNacimiento(anio);
}

@GetMapping("/estadisticas/por-curso")
public Map<String, Long> contarPorCurso() {
    return service.contarPorCurso();
}

@GetMapping("/resumen")
public List<AlumnoResumenDTO> resumenPorCurso(@RequestParam String curso) {
    return service.buscarResumenPorCurso(curso);
}
```

Pregunta de reflexión: ¿Qué devuelve el endpoint /estadisticas/por-curso?

Paso 11 - Probar las consultas

Reinicia y prueba cada endpoint:

```
# Buscar por nombre de curso
curl "http://localhost:8080/api/v1/alumnos/por-curso?nombre=5%C2%BA%20Primaria"

# Buscar por año de nacimiento
curl "http://localhost:8080/api/v1/alumnos/por-anio?anio=2010"

# Estadísticas por curso
curl "http://localhost:8080/api/v1/alumnos/estadisticas/por-curso"

# Resumen por curso
curl "http://localhost:8080/api/v1/alumnos/resumen?curso=5%C2%BA%20Primaria"
```

Observa las respuestas y los logs. Verás el SQL que Hibernate genera para cada consulta.

Pregunta de reflexión: ¿Qué diferencia hay entre el SQL generado por buscarPorNombreCurso y por buscarPorNombreCursoConCurso?

Paso 12 - Errores comunes del ejercicio

Error	Causa	Solución
Unknown entity: alumnos Named parameter no localizado	Nombre de tabla en JPQL @Param incorrecto	Usar entidad Alumno. Alinear nombres.

Error	Causa	Solución
MultipleBagFetchException	Fetch de varias listas	Cambiar modelo o separar consultas.
Filas duplicadas	Fetch de colección sin DISTINCT	Añadir DISTINCT cuando proceda.
ClassCastException en proyección	Firma DTO no coincide	Alinear tipos/orden.
Object[] difícil de exponer	Proyección sin tipo	Transformar a DTO.
LIKE no coincide	Sin comodines	Usar % con parámetros.
TransactionRequiredException	@Modifying sin transacción	Ejecutar en servicio transaccional.

Paso 13 - Reto resuelto — Consulta con subconsulta

Reto: Añadir un método que devuelva los cursos que tienen más alumnos que la media. Solución paso a paso:

Subpaso 1. Añadir el método al repositorio de cursos:

```
@Query("""
    SELECT c FROM Curso c
    WHERE SIZE(c.alumnos) > (
        SELECT AVG(SIZE(c2.alumnos)) FROM Curso c2
    )
    """)
    List<Curso> buscarCursosConMasAlumnosQueLaMedia();
```

SIZE(c.alumnos) devuelve el número de alumnos del curso.

La subconsulta (SELECT AVG(SIZE(c2.alumnos)) FROM Curso c2) calcula la media de alumnos por curso.

WHERE SIZE(c.alumnos) > (...) filtra los cursos que tienen más alumnos que la media.

Subpaso 2. Añadir el método al servicio:

```
@Transactional(readOnly = true)
public List<CursoDTO> cursosConMasAlumnosQueLaMedia() {
    return cursoRepository.buscarCursosConMasAlumnosQueLaMedia().stream()
        .map(this::toDTO)
        .toList();
}
```

Subpaso 3. Añadir el endpoint:

```
@GetMapping("/estadisticas/mas-alumnos-que-media")
public List<CursoDTO> cursosConMasAlumnosQueLaMedia() {
    return service.cursosConMasAlumnosQueLaMedia();
}
```

Subpaso 4. Probar:

curl http://localhost:8080/api/v1/cursos/estadisticas/mas-alumnos-que-media

Pregunta de reflexión: ¿Qué diferencia hay entre una subconsulta y un join?

Resultado esperado global

Al final del ejercicio, deberías tener: - Un AlumnoRepository con:

- Consultas con JOIN y JOIN FETCH.

- Funciones de string (LOWER), fecha (YEAR).
- Agregaciones (COUNT, GROUP BY, HAVING).
- Proyecciones a DTO (AlumnoResumenDTO).
- Un CursoRepository con una subconsulta.
- Un AlumnoService con los métodos correspondientes.
- Un AlumnoController con los endpoints.
- Un CursoController con el endpoint de estadísticas.

Resumen técnico

El ejercicio ha demostrado:

- JPQL: estructura, parámetros nombrados.
- Joins: JOIN, JOIN FETCH, DISTINCT.
- Funciones: LOWER, YEAR, CONCAT, SIZE.
- Agregaciones: COUNT, GROUP BY, HAVING.
- Proyecciones: devolver DTOs con new.
- Subconsultas: en el WHERE.
- Buenas prácticas: parámetros nombrados, JOIN FETCH, proyecciones.

Conclusión

En este punto 4.5 hemos profundizado en las consultas JPQL:

- JPQL: lenguaje de consulta orientado a objetos.
- Estructura: SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY.
- Parámetros: nombrados preferibles a posicionales.
- Joins: INNER JOIN, LEFT JOIN, JOIN FETCH.
- Funciones: string, numéricas, fecha, CASE WHEN.
- Agregaciones: COUNT, SUM, AVG, MIN, MAX.
- GROUP BY y HAVING:
- Proyecciones: devolver DTOs con new.
- Buenas prácticas: JOIN FETCH, DISTINCT, proyecciones, medir. En la práctica, hemos ampliado los repositorios con consultas de todos los tipos: joins, funciones, agregaciones, proyecciones y subconsultas. Hemos visto cómo se comporta cada una y cómo se integran en el servicio.

La idea clave: JPQL es una herramienta muy potente, pero hay que saber cuándo usar cada tipo de consulta. Los métodos derivados son cómodos para consultas simples; @Query para consultas personalizadas; Specifications para consultas dinámicas; proyecciones para consultas eficientes.

En el siguiente punto, 4.6 - Gestión de transacciones, profundizaremos en las transacciones con @Transactional: qué son, cómo se propagan, cómo se aíslan, y cómo se comportan ante errores.

Fin del Punto 4.5.

Práctica 4.6 - Gestión de transacciones

Contexto del ejercicio: Vamos a aplicar transacciones en el AlumnoService y el CursoService. Veremos cómo se comporta una transacción ante errores, cómo se propaga entre métodos, y cómo se configura el rollback.

Requisitos previos: Tener el proyecto mi-proyecto con las entidades Alumno y Curso del punto 4.5.

Paso 1 - Repasar las anotaciones actuales

Abre el AlumnoService y observa las anotaciones @Transactional. Ya tienes algunas de puntos anteriores:

- @Transactional(readOnly = true) en listar y consultar.
- @Transactional en crear, actualizar y eliminar.

Pregunta de reflexión: ¿Qué diferencia hay entre @Transactional y @Transactional(readOnly = true)?

Paso 2 - Provocar un rollback

Vamos a provocar un rollback para ver cómo funciona. Modifica temporalmente el método crear del servicio para que lance una excepción después de guardar:

```
@Transactional
public AlumnoResponseDTO crear(AlumnoRequestDTO request) {
    if (repositorio.existsByDni(request.getDni())) {
        throw new NegocioException(
            "Ya existe un alumno con el DNI " + request.getDni());
    }
    Curso curso = cursoRepository.findByNombre(request.getCurso())
        .orElseThrow(() -> new NegocioException("Curso no encontrado"));
    Alumno entidad = new Alumno(
        request.getNombre(),
        request.getApellidos(),
        request.getDni(),
        request.getFechaNacimiento()
    );
    entidad.setCurso(curso);
    Alumno guardado = repositorio.save(entidad);

    // Provocar un rollback
    if (request.getNombre().equals("Rollback")) {
        throw new RuntimeException("Error simulado para provocar rollback");
    }

    return toDTO(guardado);
}
```

Pregunta de reflexión: ¿Qué crees que pasará con el alumno que se ha guardado antes de la excepción?

Paso 3 - Probar el rollback

Reinicia la aplicación y prueba a crear un alumno con nombre "Rollback":

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{
    "nombre": "Rollback",
    "apellidos": "Prueba",
    "dni": "99999999Z",
    "fechaNacimiento": "2010-01-01",
    "curso": "5º Primaria"
}'
```

Verás un 500. Ahora consulta si el alumno se ha guardado:

```
curl "http://localhost:8080/api/v1/alumnos?dni=99999999Z"
```

No aparecerá. El rollback ha revertido el INSERT.

Pregunta de reflexión: ¿Qué pasaría si el método no tuviera @Transactional?

Paso 4 - Quitar el rollback temporal

Elimina el bloque que provoca el rollback:

```
// if (request.getNombre().equals("Rollback")) {
//     throw new RuntimeException("Error simulado");
// }
```

Reinicia y verifica que el alumno se crea correctamente.

Pregunta de reflexión: ¿Qué diferencia hay entre el comportamiento con y sin @Transactional?

Paso 5 - Añadir @Transactional a un nuevo método con REQUIRES_NEW

Vamos a añadir un método que registre una auditoría. Este método debe ejecutarse en una transacción independiente para que, aunque la transacción principal falle, la auditoría se guarde.

Primero, crea una entidad Auditoria:

```
@Entity
@Table(name = "auditorias")
public class Auditoria {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String operacion;

    @Column(nullable = false)
    private LocalDateTime fecha;

    @Column(length = 500)
    private String detalle;

    public Auditoria() {
        this.fecha = LocalDateTime.now();
    }

    public Auditoria(String operacion, String detalle) {
        this.operacion = operacion;
        this.detalle = detalle;
        this.fecha = LocalDateTime.now();
    }
    // getters y setters
}
```

Crea el repositorio:

```
public interface AuditoriaRepository extends JpaRepository<Auditoria, Long> {
}
```

Crea un servicio de auditoría:

```
@Service
public class AuditoriaService {

    private final AuditoriaRepository repositorio;
```

```

    public AuditoriaService(AuditoriaRepository repositorio) {
        this.repositorio = repositorio;
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void registrar(String operacion, String detalle) {
        repositorio.save(new Auditoria(operacion, detalle));
    }
}

```

Propagation.REQUIRES_NEW suspende la transacción existente y ejecuta este método en una transacción independiente.

Aunque la transacción principal falle, la auditoría se guardará.

Pregunta de reflexión: ¿Qué pasaría si la propagación fuera REQUIRED en lugar de REQUIRES_NEW?

Paso 6 - Usar el servicio de auditoría desde el servicio de alumnos

Modifica el método crear del AlumnoService para que registre una auditoría:

```

@Transactional
public AlumnoResponseDTO crear(AlumnoRequestDTO request) {
    if (repositorio.existsByDni(request.getDni())) {
        throw new NegocioException(
            "Ya existe un alumno con el DNI " + request.getDni());
    }
    Curso curso = cursoRepository.findByNombre(request.getCurso())
        .orElseThrow(() -> new NegocioException("Curso no encontrado"));
    Alumno entidad = new Alumno(
        request.getNombre(),
        request.getApellidos(),
        request.getDni(),
        request.getFechaNacimiento()
    );
    entidad.setCurso(curso);
    Alumno guardado = repositorio.save(entidad);

    auditoriaService.registrar("CREAR_ALUMNO", "DNI: " + request.getDni());

    return toDTO(guardado);
}

```

Pregunta de reflexión: ¿Por qué el método de auditoría tiene REQUIRES_NEW? ¿Qué ventaja tiene?

Paso 7 - Provocar un rollback y verificar que la auditoría persiste

Vamos a provocar un rollback después de la auditoría. Modifica el método crear:

```

@Transactional
public AlumnoResponseDTO crear(AlumnoRequestDTO request) {
    // ... código anterior
    Alumno guardado = repositorio.save(entidad);
    auditoriaService.registrar("CREAR_ALUMNO", "DNI: " + request.getDni());

    if (request.getNombre().equals("Rollback")) {
        throw new RuntimeException("Error simulado");
    }
}

```

```

    }

    return toDTO(guardado);
}

```

Pregunta de reflexión: ¿Qué pasará con el alumno y con la auditoría?

Paso 8 - Probar el rollback con auditoría

Reinicia y prueba a crear un alumno con nombre "Rollback":

```

curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{
    "nombre": "Rollback",
    "apellidos": "Prueba",
    "dni": "88888888Y",
    "fechaNacimiento": "2010-01-01",
    "curso": "5º Primaria"
  }'

```

Verás un 500. Ahora consulta la tabla de auditorías. Si tienes la consola de H2 abierta, ejecuta:

```
SELECT * FROM auditorias WHERE detalle LIKE '%88888888Y%'
```

Verás que la auditoría sí está guardada, aunque el alumno no se haya creado. El `REQUIRES_NEW` ha hecho que la auditoría se guarde en una transacción independiente.

Pregunta de reflexión: ¿Por qué la auditoría se ha guardado si la transacción principal ha fallado?

Paso 9 - Quitar el rollback temporal

Elimina el bloque que provoca el rollback. Reinicia la aplicación.

Pregunta de reflexión: ¿Qué hemos aprendido con este ejercicio?

Paso 10 - Añadir bloqueo optimista

Vamos a añadir un campo `@Version` a la entidad Alumno para el bloqueo optimista:

```

@Version
private Integer version;

```

Añade el getter y el setter.

Reinicia la aplicación. Hibernate añadirá una columna `version` a la tabla `alumnos`.

Pregunta de reflexión: ¿Qué pasa si dos transacciones intentan actualizar el mismo alumno a la vez?

Paso 11 - Probar el bloqueo optimista

Vamos a simular dos actualizaciones concurrentes. Ejecuta dos curl en paralelo:

```

# Terminal 1
curl -i -X PUT http://localhost:8080/api/v1/alumnos/1 \
  -H "Content-Type: application/json" \
  -d '{
    "nombre": "Ana",
    "apellidos": "García",

```

```
"dni": "12345678A",
"fechaNacimiento": "2010-05-12",
"curso": "5º Primaria"
}'
```

Terminal 2 (al mismo tiempo)

```
curl -i -X PUT http://localhost:8080/api/v1/alumnos/1 \
-H "Content-Type: application/json" \
-d '{
  "nombre": "Ana",
  "apellidos": "García",
  "dni": "12345678A",
  "fechaNacimiento": "2010-05-12",
  "curso": "6º Primaria"
}'
```

Una de las dos actualizaciones puede fallar con `OptimisticLockException`. Si ambas tienen éxito, es porque se han ejecutado secuencialmente (poco probable en un sistema rápido, pero posible).

Pregunta de reflexión: ¿Qué ventaja tiene el bloqueo optimista en una API REST?

Paso 12 - Errores comunes del ejercicio

Error	Causa	Solución
Cambios no persistidos	Falta transacción/entidad detached	Definir unidad de trabajo en servicio.
<code>TransactionRequiredException</code>	Modificación fuera de transacción	Añadir <code>@Transactional</code> .
Transacción no aplicada	Autoinvocación dentro del bean	Mover a otro bean/rediseñar.
<code>LazyInitializationException</code>	Uso de LAZY tras cerrar transacción	DTO dentro del servicio.
Sin rollback checked	Regla por defecto	<code>rollbackFor</code> si procede.
Sin rollback al capturar	Excepción absorbida	Relanzar/marcar rollback.
<code>OptimisticLockException</code>	Conflicto concurrente	Responder conflicto/reintentar.
Demasiadas conexiones	Abuso de <code>REQUIRES_NEW</code>	Usarlo sólo si es independiente.
Transacción larga	I/O lento dentro	Moverlo fuera.
<code>readOnly</code> mal entendido	Se trata como garantía fuerte	Usarlo como semántica/hint.

Paso 13 - Reto resuelto — Transacción con timeout

Reto: Añadir un timeout de 5 segundos a un método transaccional. Si el método tarda más, la transacción se aborta.

Solución paso a paso:

Subpaso 1. Añadir el timeout a un método:

```
@Transactional(timeout = 5)
public AlumnoResponseDTO crearConTimeout(AlumnoRequestDTO request) {
    // ...
    return crear(request);
}
```

Subpaso 2. Simular una operación lenta:

```

@Transactional(timeout = 5)
public AlumnoResponseDTO crearConTimeout(AlumnoRequestDTO request) {
    try {
        Thread.sleep(6000); // 6 segundos
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
    return crear(request);
}

```

Subpaso 3. Probar:

```

curl -i -X POST http://localhost:8080/api/v1/alumnos/con-timeout \
-H "Content-Type: application/json" \
-d '{...}'

```

Verás un error de timeout después de 5 segundos. La transacción se ha abortado.

Pregunta de reflexión: ¿En qué casos usarías un timeout en una transacción?

Resultado esperado global

Al final del ejercicio, deberías tener:

- Un AlumnoService con métodos @Transactional (lectura y escritura).
- Un AuditoriaService con @Transactional(propagation = REQUIRES_NEW).
- Una entidad Alumno con @Version para bloqueo optimista.
- La capacidad de provocar un rollback y verificarlo.
- La capacidad de usar REQUIRES_NEW para transacciones independientes.

Resumen técnico

El ejercicio ha demostrado:

- @Transactional: en métodos que modifican datos.
- readOnly = true: en consultas.
- Rollback: con RuntimeException, no con checked.
- REQUIRES_NEW: transacciones independientes.
- @Version: bloqueo optimista.
- timeout: límite de tiempo.
- Buenas prácticas: @Transactional en el servicio, métodos públicos, transacciones cortas.

Conclusión

En este punto 4.6 hemos profundizado en la gestión de transacciones:

- Transacción: unidad atómica. Propiedades ACID.
- @Transactional: gestión declarativa.
- Propagación: REQUIRED, REQUIRES_NEW, SUPPORTS, etc.
- Aislamiento: READ_COMMITTED, REPEATABLE_READ, etc.
- Bloqueo: optimista (@Version) y pesimista (@Lock).
- Rollback: RuntimeException por defecto, rollbackFor para checked.
- Buenas prácticas: transacciones en el servicio, métodos públicos, cortas.

En la práctica, hemos aplicado transacciones en el AlumnoService, hemos provocado rollbacks, hemos usado REQUIRES_NEW para una auditoría independiente, y hemos añadido bloqueo optimista con @Version.

La idea clave: las transacciones son la garantía de que la base de datos se mantiene consistente. Sin ellas, un fallo a mitad de una operación deja los datos en un estado inválido. Con ellas, todo o nada

Práctica 4.7 - Testing de JPA con @DataJpaTest

Contexto del ejercicio: Vamos a escribir tests para los repositorios AlumnoRepository y CursoRepository con @DataJpaTest. Veremos cómo preparar datos con TestEntityManager, cómo probar consultas, relaciones y constraints, y cómo verificar el comportamiento de LAZY.

Requisitos previos: Tener el proyecto mi-proyecto con las entidades Alumno y Curso del punto 4.6.

Paso 1 - Verificar la dependencia de test

Abre el pom.xml y verifica que tienes la dependencia spring-boot-starter-test:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

Esta dependencia incluye JUnit 5, Mockito, AssertJ, Spring Test y H2 (a través de spring-boot-starter-data-jpa). No necesitas añadir nada más.

Pregunta de reflexión: ¿Por qué la dependencia tiene <scope>test</scope>?

Paso 2 - Crear la estructura de paquetes de test

Los tests de repositorio van en src/test/java, en el mismo paquete que el repositorio. Si has reorganizado el proyecto por funcionalidad, el paquete es es.mecd.demo.miproyecto.alumno y es.mecd.demo.miproyecto.curso.

Crea la carpeta src/test/java/es/mecd/demo/miproyecto/alumno/ si no existe.

Pregunta de reflexión: ¿Por qué los tests se organizan en la misma estructura de paquetes que el código de producción?

Paso 3 - Crear el test básico de AlumnoRepository

Crea AlumnoRepositoryTest en src/test/java/es/mecd/demo/miproyecto/alumno/:

```
package es.mecd.demo.miproyecto.alumno;

import es.mecd.demo.miproyecto.curso.Curso;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.boot.test.autoconfigure.orm.jpa.TestEntityManager;

import java.time.LocalDate;
import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;

@DataJpaTest
class AlumnoRepositoryTest {
    @Autowired
    private TestEntityManager entityManager;

    @Autowired
```

```

private AlumnoRepository repositorio;

private Curso curso;

@BeforeEach
void setUp() {
    curso = entityManager.persistAndFlush(new Curso("5º Primaria"));
}

@Test
void guardar_debePersistirAlumno() {
    Alumno alumno = new Alumno("Ana", "García López", "12345678A",
        LocalDate.of(2010, 5, 12));
    alumno.setCurso(curso);

    Alumno guardado = repositorio.save(alumno);
    entityManager.flush();
    entityManager.clear();

    assertNotNull(guardado.getId());
    assertTrue(guardado.getId() > 0);
}
}

```

`@DataJpaTest` arranca solo la infraestructura JPA.

`@Autowired` `TestEntityManager entityManager` inyecta el gestor de entidades de test.

`@Autowired` `AlumnoRepository repositorio` inyecta el repositorio real.

`@BeforeEach` `setUp()` se ejecuta antes de cada test; prepara un curso persistido reutilizable por el caso de prueba.

`persistAndFlush` persiste la entidad y hace flush. Así el curso tiene ID antes de que empiece el test.

`entityManager.flush()` fuerza el INSERT del alumno en la base de datos.

`entityManager.clear()` limpia el contexto para que la próxima consulta lea de la base de datos.

Pregunta de reflexión: ¿Por qué se hace `entityManager.clear()` antes de verificar la persistencia?

Paso 4 - Probar el método `findByCurso`

Añade el test:

```

@Test
void findByCurso_debeDevolverAlumnosDelCurso() {
    Curso otroCurso = entityManager.persistAndFlush(new Curso("6º Primaria"));

    Alumno ana = new Alumno("Ana", "García", "12345678A", null);
    ana.setCurso(curso);
    entityManager.persistAndFlush(ana);

    Alumno luis = new Alumno("Luis", "Pérez", "87654321B", null);
    luis.setCurso(curso);
    entityManager.persistAndFlush(luis);

    Alumno maria = new Alumno("María", "López", "11111111C", null);
}

```

```

        maria.setCurso(otroCurso);
        entityManager.persistAndFlush(maria);

        List<Alumno> resultado =
            repositorio.findByCursoNombreIgnoreCaseAndEliminadoFalse(
                "5º Primaria");

        assertEquals(2, resultado.size());
        assertTrue(resultado.stream()
            .allMatch(a -> a.getCurso().getNombre()
                .equals("5º Primaria")));
    }

```

El test prepara tres alumnos: dos en 5º Primaria y uno en 6º Primaria. Llama al método derivado por la propiedad anidada `curso.nombre` y verifica que devuelve dos.

Pregunta de reflexión: ¿Qué pasaría si el método `findByCurso` devolviera los tres alumnos? ¿Qué indicaría eso?

Paso 5 - Probar el caso vacío

Añade el test:

```

@Test
void findByCurso_debeDevolverListaVacía_cuandoNoHayAlumnos() {
    List<Alumno> resultado =
        repositorio.findByCursoNombreIgnoreCaseAndEliminadoFalse(
            "Curso Inexistente");

    assertNotNull(resultado);
    assertTrue(resultado.isEmpty());
}

```

El test consulta por `curso.nombre` con un curso inexistente y verifica que devuelve una lista vacía, no null.

Pregunta de reflexión: ¿Por qué es importante verificar que devuelve [] y no null?

Paso 6 - Probar el método `existsByDni`

Añade el test:

```

@Test
void existsByDni_debeDevolverTrue_cuandoExiste() {
    Alumno ana = new Alumno("Ana", "García", "12345678A", null);
    ana.setCurso(curso);
    entityManager.persistAndFlush(ana);

    boolean existe = repositorio.existsByDni("12345678A");

    assertTrue(existe);
}

@Test
void existsByDni_debeDevolverFalse_cuandoNoExiste() {
    boolean existe = repositorio.existsByDni("99999999Z");

    assertFalse(existe);
}

```

El primer test verifica que `existsByDni` devuelve true cuando el DNI existe. El segundo, que devuelve false cuando no existe.

Pregunta de reflexión: ¿Qué diferencia hay entre `existsByDni` y `findByDni`?

Paso 7 - Probar el constraint unique del DNI

Añade el test:

```
@Test
void guardar_debeLanzarExcepcion_cuandoDniDuplicado() {
    Alumno ana = new Alumno("Ana", "García", "12345678A", null);
    ana.setCurso(curso);
    entityManager.persistAndFlush(ana);

    Alumno duplicado = new Alumno("Otro", "Alumno", "12345678A", null);
    duplicado.setCurso(curso);

    assertThrows(DataIntegrityViolationException.class, () -> {
        repositorio.saveAndFlush(duplicado);
    });
}
```

El test persiste un alumno con DNI 12345678A. Luego intenta persistir otro con el mismo DNI. La base de datos debe lanzar `DataIntegrityViolationException` porque el campo `dni` tiene `unique = true`.

`saveAndFlush` guarda y fuerza el flush inmediato. Sin el flush, la excepción podría no lanzarse hasta el commit (que en el test es al final).

Pregunta de reflexión: ¿Qué diferencia hay entre `save` y `saveAndFlush`? ¿Cuándo usarías cada uno?

Paso 8 - Probar una consulta con @Query

Añade el test:

```
@Test
void buscarPorAnioNacimiento_debeFiltrarPorAnio() {
    Alumno ana = new Alumno(
        "Ana", "García", "12345678A",
        LocalDate.of(2010, 5, 12));
    ana.setCurso(curso);
    entityManager.persistAndFlush(ana);

    Alumno luis = new Alumno(
        "Luis", "Pérez", "87654321B",
        LocalDate.of(2009, 9, 3));
    luis.setCurso(curso);
    entityManager.persistAndFlush(luis);

    Alumno maria = new Alumno(
        "María", "López", "11111111C",
        LocalDate.of(2011, 3, 20));
    maria.setCurso(curso);
    entityManager.persistAndFlush(maria);

    List<Alumno> resultado = repositorio.buscarPorAnioNacimiento(2010);
}
```

```

    assertEquals(1, resultado.size());
    assertEquals("Ana", resultado.get(0).getNombre());
}

```

El test prepara tres alumnos con fechas distintas. Llama a buscarPorAnioNacimiento(2010) y verifica que solo devuelve al alumno nacido en 2010.

Pregunta de reflexión: ¿Qué pasaría si el método devolviera los tres alumnos? ¿Qué indicaría eso?

Paso 9 - Probar la relación entre Alumno y Curso

Añade el test:

```

@Test
void guardar_debePersistirLaRelacionConCurso() {
    Alumno alumno = new Alumno("Ana", "García", "12345678A", null);
    alumno.setCurso(curso);

    Alumno guardado = repositorio.save(alumno);
    entityManager.flush();
    entityManager.clear();

    Alumno recuperado = repositorio.findById(guardado.getId()).orElseThrow();
    assertNotNull(recuperado.getCurso());
    assertEquals("5º Primaria", recuperado.getCurso().getNombre());
}

```

El test persiste un alumno con su curso, limpia el contexto, y recupera el alumno desde la base de datos. Verifica que la relación se ha persistido.

Sin entityManager.clear(), Hibernate podría devolver la entidad del caché, que tiene la relación en memoria. El test pasaría aunque la relación no se hubiera persistido. Con clear(), la lectura va a la base de datos y el test es fiable.

Pregunta de reflexión: ¿Qué pasaría si el @JoinColumn estuviera mal configurado? ¿El test lo detectaría?

Paso 10 - Crear el test de CursoRepository

Crea CursoRepositoryTest en src/test/java/es/mecd/demo/miproyecto/curso/:

```

package es.mecd.demo.miproyecto.curso;

import es.mecd.demo.miproyecto.alumno.Alumno;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.boot.test.autoconfigure.orm.jpa.TestEntityManager;
import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;

@DataJpaTest
class CursoRepositoryTest {

    @Autowired
    private TestEntityManager entityManager;
}

```

```

@Autowired
private CursoRepository repositorio;

@Test
void findByNombre_debeDevolverCurso_cuandoExiste() {
    entityManager.persistAndFlush(new Curso("5º Primaria"));

    Optional<Curso> resultado = repositorio.findByNombre("5º Primaria");

    assertTrue(resultado.isPresent());
    assertEquals("5º Primaria", resultado.get().getNombre());
}

@Test
void findByNombre_debeDevolverVacio_cuandoNoExiste() {
    Optional<Curso> resultado = repositorio.findByNombre("Inexistente");

    assertTrue(resultado.isEmpty());
}

@Test
void guardarConAlumnos_debePersistirLaRelacionOneToMany() {
    Curso curso = new Curso("5º Primaria");
    curso.addAlumno(new Alumno("Ana", "García", "12345678A", null));
    curso.addAlumno(new Alumno("Luis", "Pérez", "87654321B", null));

    entityManager.persistAndFlush(curso);
    entityManager.clear();

    Curso recuperado = entityManager.find(Curso.class, curso.getId());

    assertEquals(2, recuperado.getAlumnos().size());
}
}

```

El primer test verifica que findByNombre devuelve el curso. El segundo, que devuelve vacío si no existe. El tercero, que la relación @OneToMany persiste correctamente.

Pregunta de reflexión: ¿Por qué el test de findByNombre devuelve Optional en lugar de Curso?

Paso 11 - Ejecutar los tests

Ejecuta los tests desde la terminal:

```

./mvnw test -Dtest=AlumnoRepositoryTest
./mvnw test -Dtest=CursoRepositoryTest

```

O todos los tests:

```
./mvnw test
```

Verás algo como:

```

Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS

```

Si algún test falla, el informe muestra cuál y por qué.

Pregunta de reflexión: ¿Qué ventaja tiene que los tests de JPA se ejecuten con H2 en memoria?

Paso 12 - Errores comunes del ejercicio

Error	Causa	Solución
LazyInitializationException en test	Fuera del contexto transaccional	Revisar slice/fetch.
No se lanza constraint	No hay flush	Usar saveAndFlush().
Falso positivo de relación	Primer nivel de caché	flush() + clear().
NoSuchBeanDefinitionException	Slice incorrecto	Usar @DataJpaTest.
Test lento	@SpringBootTest innecesario	Usar slice JPA.
Fallo por orden	Sin ORDER BY	Comparar conjunto/definir orden.
Contaminación entre tests	Aislamiento desactivado	Mantener rollback por test.
Entidad sin ID observable	No se sincronizó	persistAndFlush() si hace falta.
findById vacío	No persistido/flush pendiente	Persistir y flush.
assertThrows falla	Excepción ocurre en otro punto/tipo	Forzar operación exacta y verificar tipo.

Paso 13 - Reto resuelto — Test para Specification

Reto: Escribir un test para una Specification que combine dos filtros: por curso y por estado.

Solución paso a paso:

Subpaso 1. Añadir el test a AlumnoRepositoryTest:

```
@Test
void spec_debeFiltrarPorCursoYEstado() {
    Curso otroCurso = entityManager.persistAndFlush(new Curso("6º Primaria"));
    Alumno ana = new Alumno("Ana", "García", "12345678A", null);
    ana.setCurso(otroCurso);
    ana.setEstado(EstadoAlumno.ACTIVO);
    entityManager.persistAndFlush(ana);

    Alumno luis = new Alumno("Luis", "Pérez", "87654321B", null);
    luis.setCurso(otroCurso);
    luis.setEstado(EstadoAlumno.ACTIVO);
    entityManager.persistAndFlush(luis);

    Alumno maria = new Alumno("María", "López", "11111111C", null);
    maria.setCurso(otroCurso);
    maria.setEstado(EstadoAlumno.INACTIVO);
    entityManager.persistAndFlush(maria);

    Specification<Alumno> spec = Specification.allOf(
        AlumnoSpecifications.activos(),
        AlumnoSpecifications.curso("5º Primaria"),
        AlumnoSpecifications.estado(EstadoAlumno.ACTIVO));

    List<Alumno> resultado = repositorio.findAll(spec);

    assertEquals(1, resultado.size());
    assertEquals("Ana", resultado.get(0).getNombre());
}
```

Subpaso 2. Verificar que la Specification devuelve solo la alumna que cumple ambos filtros (Ana).

Subpaso 3. Ejecutar el test:

```
./mvnw test -Dtest=AlumnoRepositoryTest#spec_debeFiltrarPorCursoYEstado
```

Pregunta de reflexión: ¿Por qué el test prepara tres alumnos y espera solo uno?

Resultado esperado global

Al final del ejercicio, deberías tener:

- Un AlumnoRepositoryTest con:
 - Test de guardar.
 - Test de findByCurso (éxito y caso vacío).
 - Test de existsByDni (true y false).
 - Test de DNI duplicado (DataIntegrityViolationException).
 - Test de @Query (buscarPorAnioNacimiento).
 - Test de relación con Curso.
 - Test de Specification.
- Un CursoRepositoryTest con:
 - Test de findByNombre (éxito y vacío).
 - Test de relación @OneToMany. Todos los tests pasando con ./mvnw test.

Resumen técnico

El ejercicio ha demostrado:

- @DataJpaTest: arranca solo la infraestructura JPA.
- TestEntityManager: para preparar datos.
- persistAndFlush: persistir y forzar el flush.
- clear(): limpiar el contexto para leer de la base de datos.
- Probar consultas derivadas: findByCurso, existsByDni.
- Probar @Query: buscarPorAnioNacimiento.
- Probar Specifications.
- Probar relaciones: @ManyToOne, @OneToMany.
- Probar constraints: DataIntegrityViolationException.
- Buenas prácticas: nombres descriptivos, caso vacío, clear().

Conclusión

En este punto 4.7 hemos aprendido a testear JPA con @DataJpaTest: